
ACKNOWLEDGEMENTS

I would like to thank everyone who directly or indirectly contributed to the development of this work and who lent me a hand in carrying out this project and motivated me throughout my studies. I would like to express my sincere thanks as well as my deep gratitude to my end-of-study project supervisor Mr. Mouhieddine BELGUTH for the trust he placed in me throughout this work as well as for his support, advices, encouragements and his guidance. My thanks are also to the members of jury who accepted to honor me with their presence and evaluate my

work,
Ghassen 

CONTENTS

General Introduction	1
1 State of the Art	3
1.1 Presenting the host company:	3
1.2 Preliminary study	4
1.2.1 Study of existing	4
1.2.1.1 analysis of the existing	4
1.2.1.2 Critique of the existing:	5
1.2.2 Suggested solution	6
1.3 Requirement specification	6
1.3.1 Identifying major actors in the system	6
1.3.2 Functional requirements	7
1.3.3 Non-functional requirements	8
1.4 Working Methodology	9
1.5 Gantt Diagram	10
2 conceptual study	11
2.1 Design methodology	11
2.2 Use case diagrams	12
2.2.1 Use case diagram of the administrator	12
2.2.1.1 Global use case diagram	12

2.2.2	Use case diagram of the merchandiser	23
2.2.2.1	Global use case diagram	23
2.3	Class diagram	27
2.4	Sequence diagram	28
2.4.1	Sequence diagram of “Login”	28
2.4.2	Sequence diagram of “create user”	29
2.4.3	Sequence diagram of “create a leave request”	30
3	Realisation	32
3.1	System architecture	32
3.1.1	N-tier architecture	32
3.1.2	MVC pattern	33
3.2	Technology choices	34
3.2.1	Spring Boot	34
3.2.2	React	35
3.2.3	Database	35
3.3	Development environment and tools	36
3.3.1	Visual Studio Code	36
3.3.2	Spring Tool Suite	36
3.3.3	Postman	37
3.3.4	GitLab	37
3.3.5	Draw.io	38
3.3.6	Beekeeper Studio	38
3.4	Interfaces	39
3.4.1	Administrator	39
3.4.1.1	Login	39
3.4.1.2	Categories management	40
3.4.1.3	Planning management	44
3.4.1.4	Leave requests management	48
3.4.1.5	Claims management	49
3.4.1.6	Reports management	51

3.4.2	Merchandiser : (Developing the mobile app is the task of colleague) . . .	53
3.4.2.1	Login	53
3.4.2.2	Locating a GMS	53
3.4.2.3	Creating a report	54

Conclusion& Perspective	56
------------------------------------	-----------

Annex A	60
----------------	-----------

LIST OF FIGURES

1.1	Logo «TeamDev»	3
1.2	Etude de l'existant«Foko Retail»	5
1.3	Etude de l'existant«Pepperi mobile merchandising»	5
1.4	V-life cycle model	9
1.5	Gantt diagram	10
2.1	global use case for the administrator	12
2.2	user accounts management	13
2.3	Use case diagram of “competitors’management”:	14
2.4	Use case diagram of “Claim management”	15
2.5	Use case diagram of “GMS management”	16
2.6	Use case diagram of “Products management”	17
2.7	Use case diagram of “Leave requests management”	18
2.8	Use case diagram of “Planning management”	19
2.9	Use case diagram of “Planning management”	20
2.10	Use case diagram of “Product category management”	21
2.11	Use case diagram of “claim type management”	22
2.12	Use case diagram of the merchandiser	23
2.13	Use case diagram of “Establishing a report”	24
2.14	Use case diagram of “Establishing a report”	25
2.15	Use case diagram of “Using the messaging service”	26

2.16	Class diagram	27
2.17	Sequence diagram of “Login”	28
2.18	Sequence diagram of “create user”	29
2.19	Sequence diagram of “create a leave request”	30
3.1	Tier architecture	33
3.2	Interaction within MVC	34
3.3	Spring Boot	34
3.4	React	35
3.5	PostgreSQL	35
3.6	Visual Studio Code	36
3.7	Spring Tool Suite	36
3.8	Postman	37
3.9	GitLab	37
3.10	Draw.io	38
3.11	Beekeeper Studio	38
3.12	Interface login	39
3.13	Interface lthe list of categories	40
3.14	Interface Add a new category	41
3.15	Interface Update a category	42
3.16	Interface Delete a category	43
3.17	Interface merchandiser’s planning and advancement	44
3.18	Interface new planning	45
3.19	Interface Update a planning	46
3.20	Interface Delete a planning	47
3.21	Interface Managing leave requests list	48
3.22	Interface list of claims	49
3.23	Interface claim details	50
3.24	Interface Delete a Claim	51
3.25	Interface list of reports	51
3.26	Interface report details	52

3.27 Interface event Before After in report details 52

3.28 Login & HomePage 53

3.29 Locating a GMS to start a report 54

3.30 Creating a report and populating it with events 55

GENERAL INTRODUCTION

Decision making is one of the most critical processes for any enterprise, entrepreneurs make daily decisions about almost everything. Some of those are rather simple with minimum effects on the company but others could influence the overall business processes, those particular ones need enough informations to make sure that the right decision is being taken, but in reality, this is not always the case.

In a survey done by "BI-survey.com" 50% of entrepreneurs claim that they make decisions without the necessary informations, 40% claim that the quality of data they get is not good enough and other 32% claim that the relevant data is too expensive to get. [1]

At a time when competitive intensity is raging, manufacturers and distributors are competing to gain new shares in local markets. In order to ensure a good share several companies employ agents on the ground which have a main task to collect data about the market, tailor local marketing campaigns based on area trends and ensure product appearance and supply in various stores throughout their designated geographic area and that's exactly the job of a merchandiser.

Despite how crucial the merchandiser's job is in collecting data and ensuring local operations including sales, supply and marketing most of the work is currently done manually, decreasing the efficiency of the merchandiser and the quality of collected data.

¹Introduction

This end of studies project aims to solve those issues facing the merchandising team by conceiving and developing a web and mobile application called "Merchandising Team".

To attain our goal we divided this internship report into 3 major parts:

- First chapter "State of the art» this chapter focuses on the analysis of the existing solutions, the functional and non-functional needs and specifying the working method.
- Second chapter "Detailed Conception of the application" in which a detailed conception is given, a major step before implementing our solution.
- Third chapter "Realisation of the project», in this chapter a justification about the choice of technologies used is provided alongside with screenshots of our application.

Finally, we conclude our work with a general conclusion in which we discuss the future perspectives of our solution and the potential changes we intend to implement in order to further improve it and make it more efficient for the merchandisers.

CHAPTER 1

STATE OF THE ART

Introduction

The state of the art is the explicit knowledge about software engineering which is documented from various sources. First of all we will start by presenting the host company.

1.1 Presenting the host company:

TeamDev is a local software company founded in 2018 responsible for developing and maintaining a variety of IT projects for its clients. Its services are focused mainly on web and mobile solutions.



Figure 1.1: Logo «TeamDev»

¹State of the Art

1.2 Preliminary study

Preliminary system study is the first stage of system development life cycle in which we study the viability of the project. First of all we start by a study of existing phase where we analyse the existing solutions on the market, their drawbacks and suggest a more suitable solution for the user's needs.

1.2.1 Study of existing

To ensure the quality of the system, conducting a study of existing is crucial .This phase allows us to analyse the existing solutions, their pros and cons and make sure we avoid those drawbacks in our solution.

1.2.1.1 analysis of the existing

In the this part we take a thorough look at the various existing solutions in the market ,focusing on their advantages and disadvantages . The solutions we're conducting our study upon are:

«**Foko Retail**»: is a software that allows merchandisers and managers to perform retail audits and visual merchandising inspections remotely, whether by smartphone or tablet. In this application the user could assign merchandising tasks, collect photos of displays, give feedback, and monitor the progress of the field team's day-to-day duties.

²State of the Art

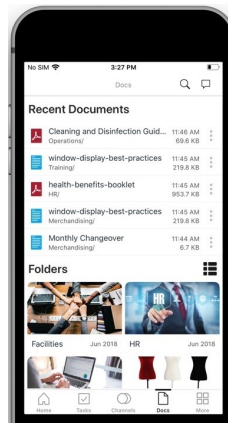


Figure 1.2: Etude de l'existant «Foko Retail»

«**Pepperi mobile merchandising**»: is a mobile software to equip the merchandising field teams with a tool able to manage merchandising activities and collect data about local markets.

Activity List						
From: 22/09/2016 To: 22/03/2017						
Activity Number	Type	Date	Account Name	Sales Rep	Shelf Photo taken on site	Shelf photo
33009459	Retail Audit	22/03/2017, 4:13 PM	A Las Vegas Store 4B	Mary Merchandizer	☐	
33014389	Retail Audit	22/03/2017, 4:12 PM	A Las Vegas Store D	Dana Morris	☒	
32919702	Retail Audit	22/03/2017, 4:07 PM	NYstore404	Dana Morris	☒	
33014397	Retail Audit	22/03/2017, 4:06 PM	A Las Vegas Store 3C	Dana Morris	☒	
32919707	Retail Audit	22/03/2017, 4:05 PM	NYstore267	Dana Morris	☒	
33009181	Retail Audit	22/03/2017, 4:05 PM	A Las Vegas Store 4B	Mary Merchandizer	☒	
32919705	Retail Audit	22/03/2017, 3:54 PM	NYstore397	Dana Morris	☒	
33014374	Retail Audit	22/03/2017, 3:47 PM	A Las Vegas Store 1A	Dana Morris	☒	
32919706	Retail Audit	19/03/2017, 3:24 PM	NYstore318	Dana Morris	☐	
32919683	Retail Audit	19/03/2017, 3:22 PM	NYstore758	Dana Morris	☐	

Figure 1.3: Etude de l'existant «Pepperi mobile merchandising»

1.2.1.2 Critique of the existing:

Those available solutions have a variety of pros and cons which include:

«**Foko Retail**»: is equipped with a powerful notification system that keeps merchandisers up to date with changes throughout the company and other pertinent informations. This application offers the ability to reach out to anyone in the company anytime easily.

Foko retail doesn't offer its users (merchandisers) a leave request tool , competitors related data or a map to locate the target stores.

³State of the Art

«**Pepperi** »: is a powerful, user-friendly tool equipped with a map, route planning, monitoring the field teams effectively in a centralized way.

This application does not contain a dashboard for the administrator and does not offer any data about the competitors. Moreover the events supported by this application are limited to tracking the shelf's state.

1.2.2 Suggested solution

We suggest a multiplatform based solution (web application for the administrator and a mobile application for merchandisers) which could digitalise the merchandiser's job.

This solution should support multiple events, a dashboard with statistics for the administrator, an ergonomic, and simple to use interface and collects data about competitors and their activities which could be used to plan marketing campaigns.

Our application should contain a map to guide merchandisers to their target supermarkets and allow them to communicate with each other using a messaging service, moreover they should be able to send claims and leave requests.

1.3 Requirement specification

A software requirements specification is the description of the system to be developed and its functional and non-functional requirements. It forms the basis for any agreement between customers and contractors on the way the software product should function.

1.3.1 Identifying major actors in the system

The administrator is a major actor in our solution, its main duties consist of the management of products, competitor, leave request, claims and creating the planning for each merchandiser.

The second actor is the merchandiser whose main duties are to follow the planning and create reports for each visited GMS.

⁴State of the Art

1.3.2 Functional requirements

Our solution should contain the following requirements:

the administrator:

- The administrator is the actor tasked with creating user accounts, updating those accounts when needed and deleting them.
- The administrator has the possibility of creating, updating and deleting any target supermarket in the system and have full access to its data.
- Using his interface, the administrator is able to add new products, updating and deleting them.
- The administrator manages daily and monthly schedule of all merchandisers.
- The administrator receive notifications regarding the merchandiser's actions, reports or messaging service notifications.
- All reclamations are reported to the administrator who could list them.
- The administrator could use the messaging service to communicate with his fellow merchandisers.
- The administrator receives leave requests from merchandisers and has the possibility to either accept or reject them.

the merchandiser:

- The merchandiser could report a reclamation to the administrator.
- Merchandisers have the ability to send leave requests.
- Merchandisers could do the referencing of the company products inside a particular supermarket.
- Merchandisers could use the messaging service to communicate with other merchandisers or the administrator.
- Merchandisers could follow a daily planning and see their monthly planning.
- Merchandisers should send a report for every visit to supermarket they made and they could see the target supermarkets on a map.

⁵State of the Art

1.3.3 Non-functional requirements

Non-functional requirements define system attributes which are mostly hidden from the end user.

Our application should obey to the following rules:

- **Ergonomics:** this solution should be user friendly with an attractive design and we will ensure that by using some famous React UI frameworks such as “Material UI”.
- **Maintainability:** while coding we try as much as possible to stick to the best practises of software development and design patterns to ensure an easy maintainability of our product.
- **Security:** By logging to the server, the user of the application (both administrator and merchandiser) acquire a JWT token that has to be send with every call to the backend to ensure his identity.
- **Interoperability:** we use technologies that ensure our product could be used on various platforms and operation systems (React Native for mobile, spring boot for backend and react for web from a single codebase).

⁶State of the Art

1.4 Working Methodology

When developing our solution, we must take into consideration all necessary constraints of software development such as: ergonomics of user interface, performance issues, etc. that's why we must comply with certain rules in order to ensure the quality of the software and deadlines. This is where the life cycle comes in handy which includes all stages from design to development.

We choose to use the V-Model which is a model where execution of processes happens in a sequential manner in a V-shape and based on the association of a testing phase for each corresponding development stage.

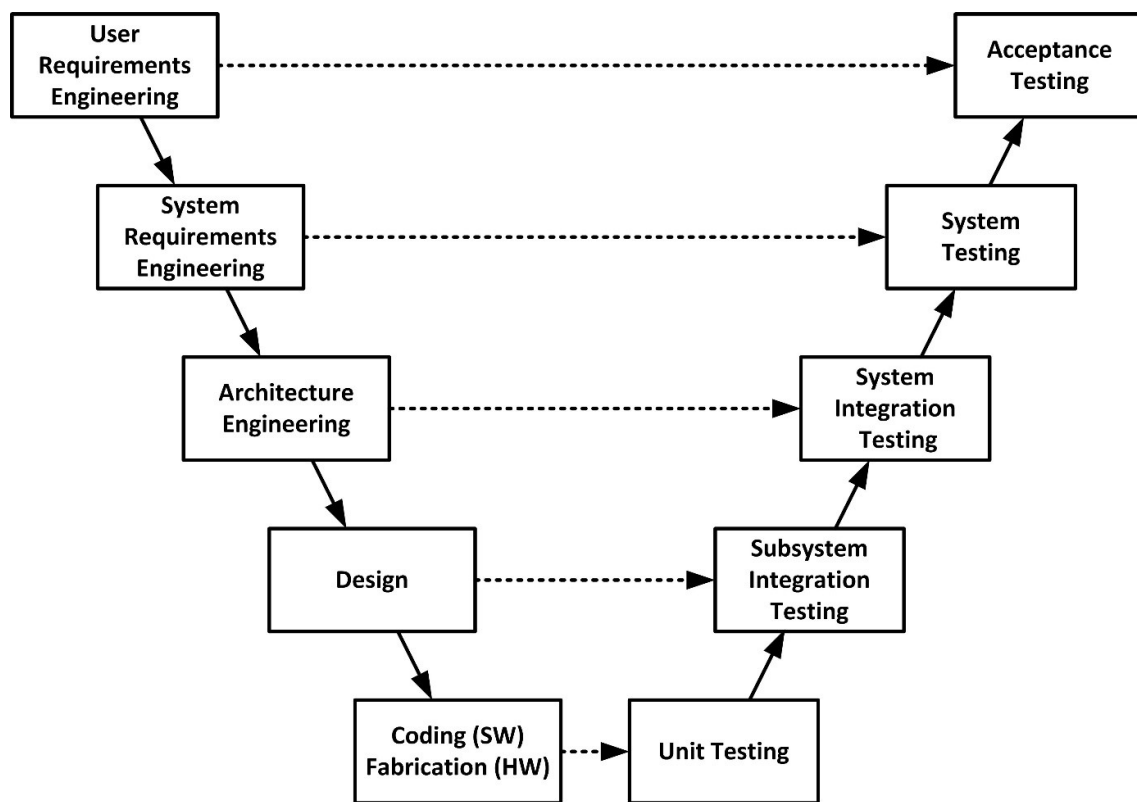


Figure 1.4: V-life cycle model

⁷State of the Art

1.5 Gantt Diagram

This work was carried out over a period of 4 months. The distribution of tasks throughout that duration is illustrated by the following Gantt diagram in the following figure.

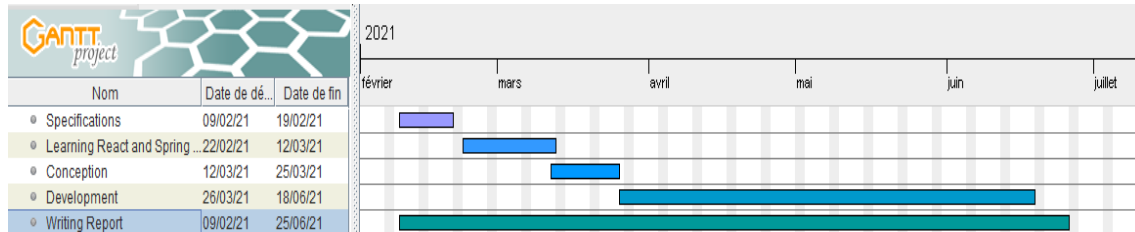


Figure 1.5: Gantt diagram

Conclusion

In this chapter we performed the study of existing by searching and studying the already existing solutions in the market, suggested our own, specified major actors alongside with both functional and non-functional requirements and finally targeted our working methodology . The next chapter will dive into detailed conception of our solution.

⁸State of the Art

CHAPTER 2

CONCEPTUAL STUDY

Introduction

Conception is the most critical phase when it comes to software development in which an overall design of the application is specified and adapted.

In this chapter we start by specifying the design methodology, various global and detailed diagrams and we will finish by a conclusion.

2.1 Design methodology

To design our system we choose to use UML (Unified modeling Language) as an object oriented modeling language.

UML is a general purpose object oriented modeling language which uses various types of diagrams to provide a standard graphical language to visualize the design of a system by specifying its structure, behavior and interactions.

UML helps to simplify complex software design by reducing lots of words of explanation to a few graphical, more understandable diagrams.

¹conceptual study

2.2 Use case diagrams

Use case diagrams provide a higher level view of the system by showing a graphical representation of a user's various interactions.

Users are represented as actors interacting with use cases represented by circles.

2.2.1 Use case diagram of the administrator

2.2.1.1 Global use case diagram

The figure below presents a global use case for the administrator in our application.

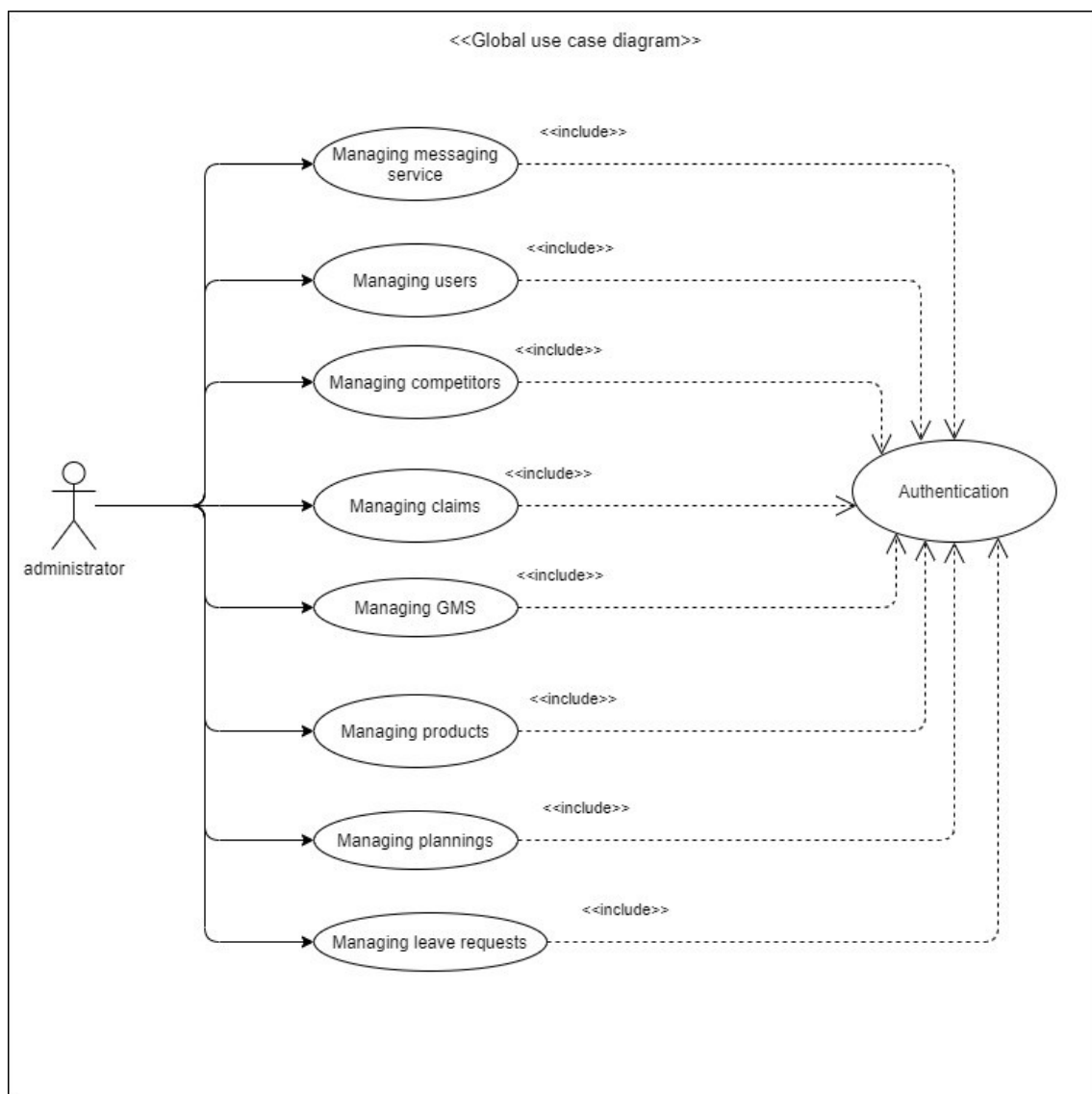


Figure 2.1: global use case for the administrator

²conceptual study

Use case diagram of “user accounts management”:

In the following use case diagram we detailed different possible actions that could be done by the administrator to ensure his management of user accounts.

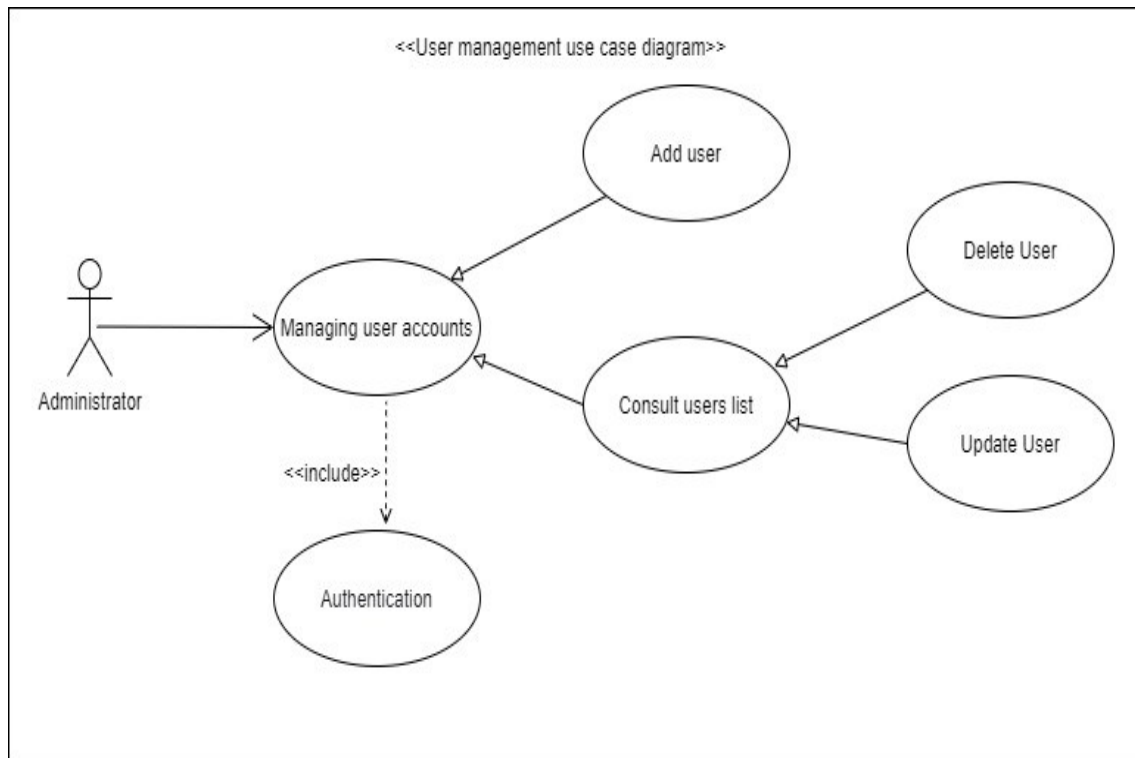


Figure 2.2: user accounts management

After authentication the user who has the role of administrator could manage the user accounts. He has the possibility to add a new user to the system or consult the list of users from which he could either delete a user or update his data.

³conceptual study

Use case diagram of “competitors’ management”:

In this use case diagram we could see the various possible actions of the admin in order to manage competitors.

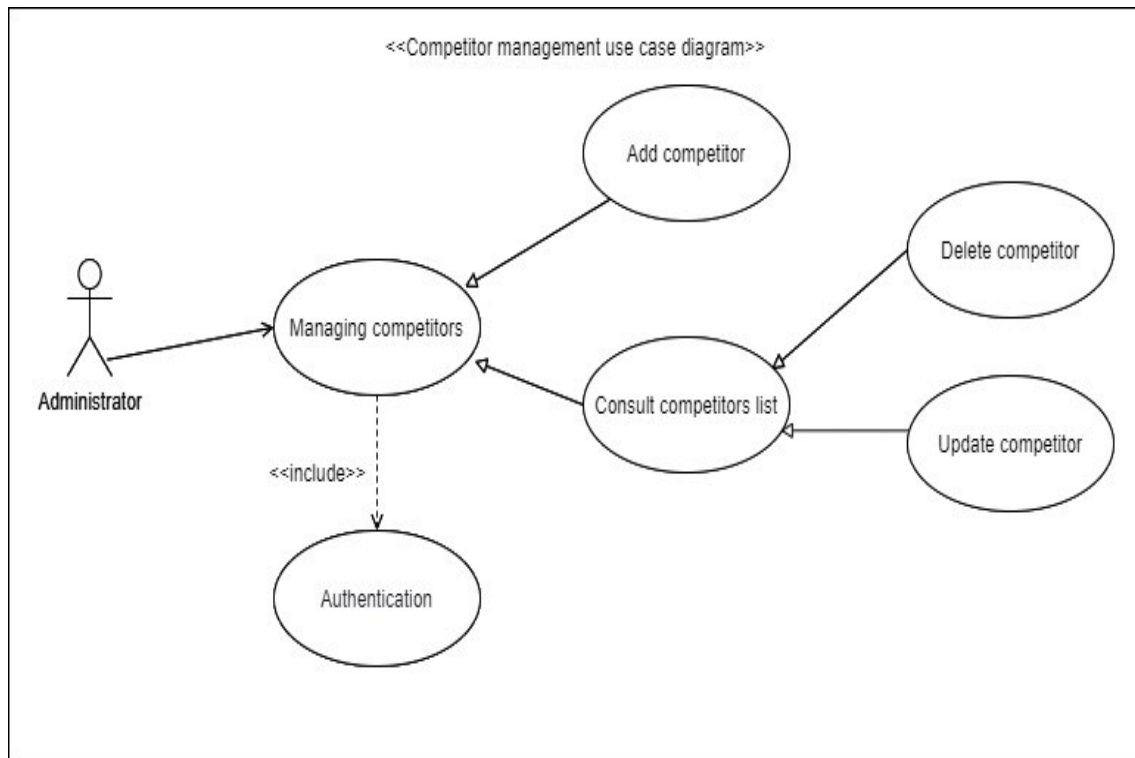


Figure 2.3: Use case diagram of “competitors’management”:

After being authenticated, the administrator can manage competitors, he has the possibility to add a new competitor, consult the list of competitors from which he could delete any competitor or update its relative data.

⁴conceptual study

Use case diagram of “Claim management”:

In the following diagram we could observe the various actions of the admin in order to manage claims.

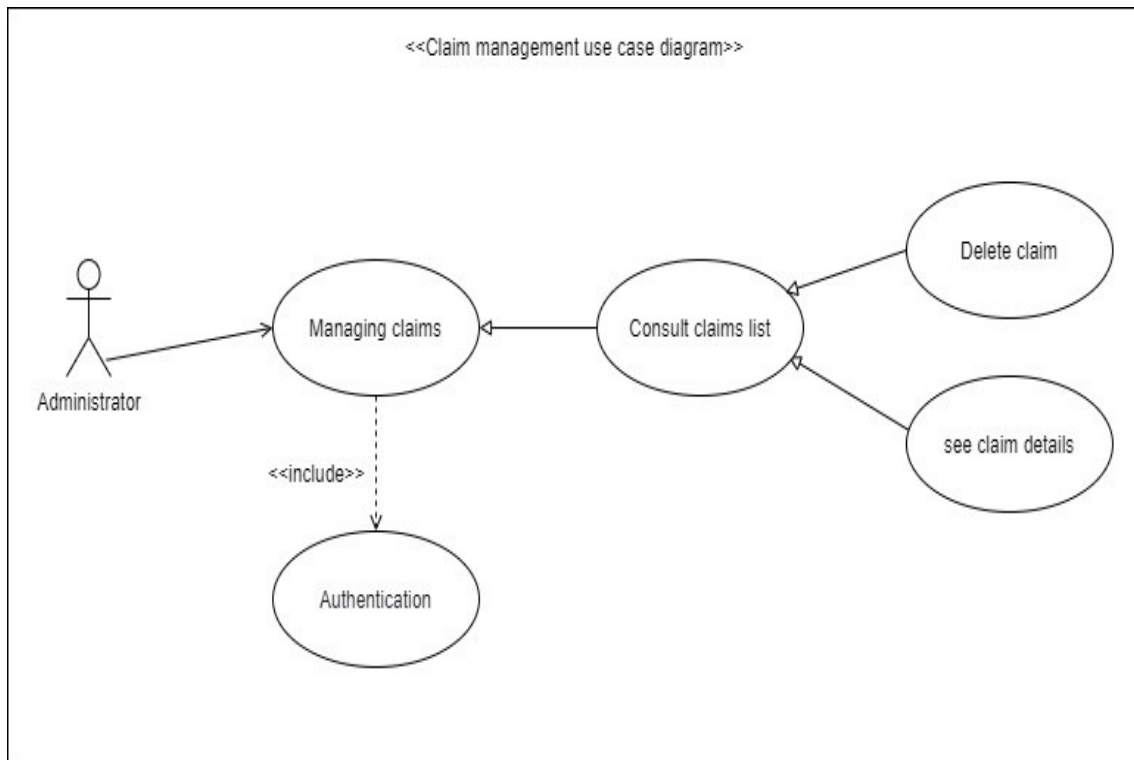


Figure 2.4: Use case diagram of “Claim management”

If he is authenticated, the administrator could manage claims sent by merchandisers By consulting the list of those claims. That list gives him the possibility to see the details of each claim or delete it.

Use case diagram of “GMS management”:

GMS stands in French for (Grande et Moyenne Surface) and by that we mean the different types of relatively big and medium supermarkets. In this use case we explain possible actions by the administrator in order to manage it.

⁵conceptual study

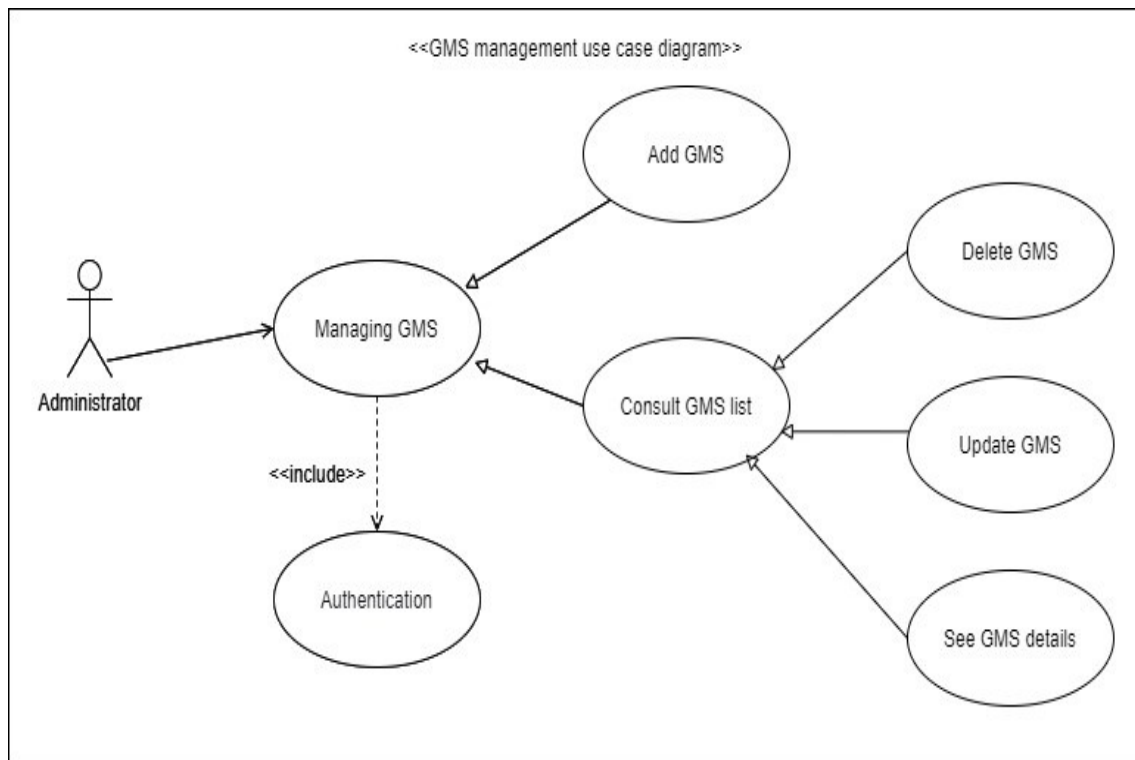


Figure 2.5: Use case diagram of “GMS management”

After authentication , the administrator has the possibility to manage GMS .he could add a new GMS or consult the overall list of GMS from which he could either see GMS details, Delete GMS or update it.

Use case diagram of “Products management”:

In the following use case diagram we detail possible actions that could be done by the administrator to manage products.

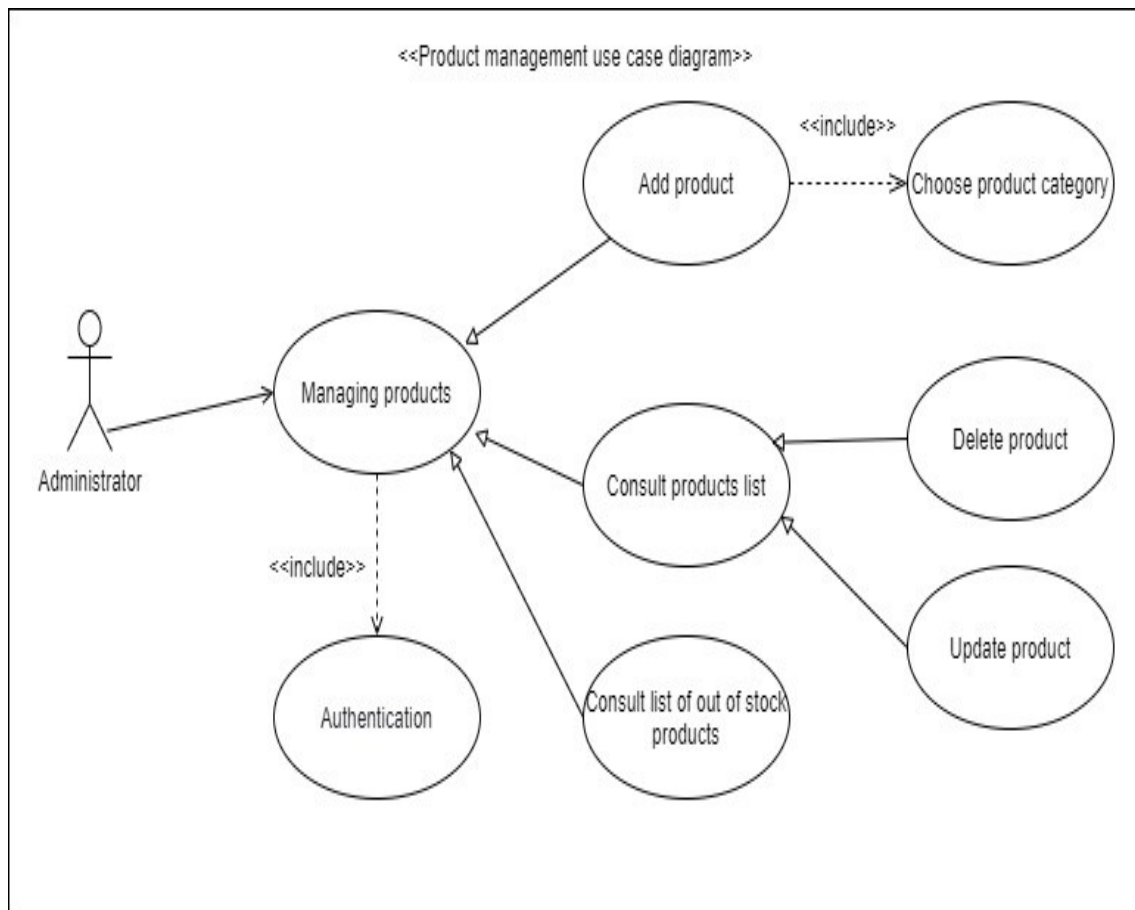


Figure 2.6: Use case diagram of “Products management”

After being authenticated, the administrator is able to manage products, he could either add a new product or consult the products list from which he has the possibility to delete a product or update it.

Use case diagram of “Leave requests management”:

In this use case diagram, we explain possible actions that could be done by the administrator to ensure leave requests management.

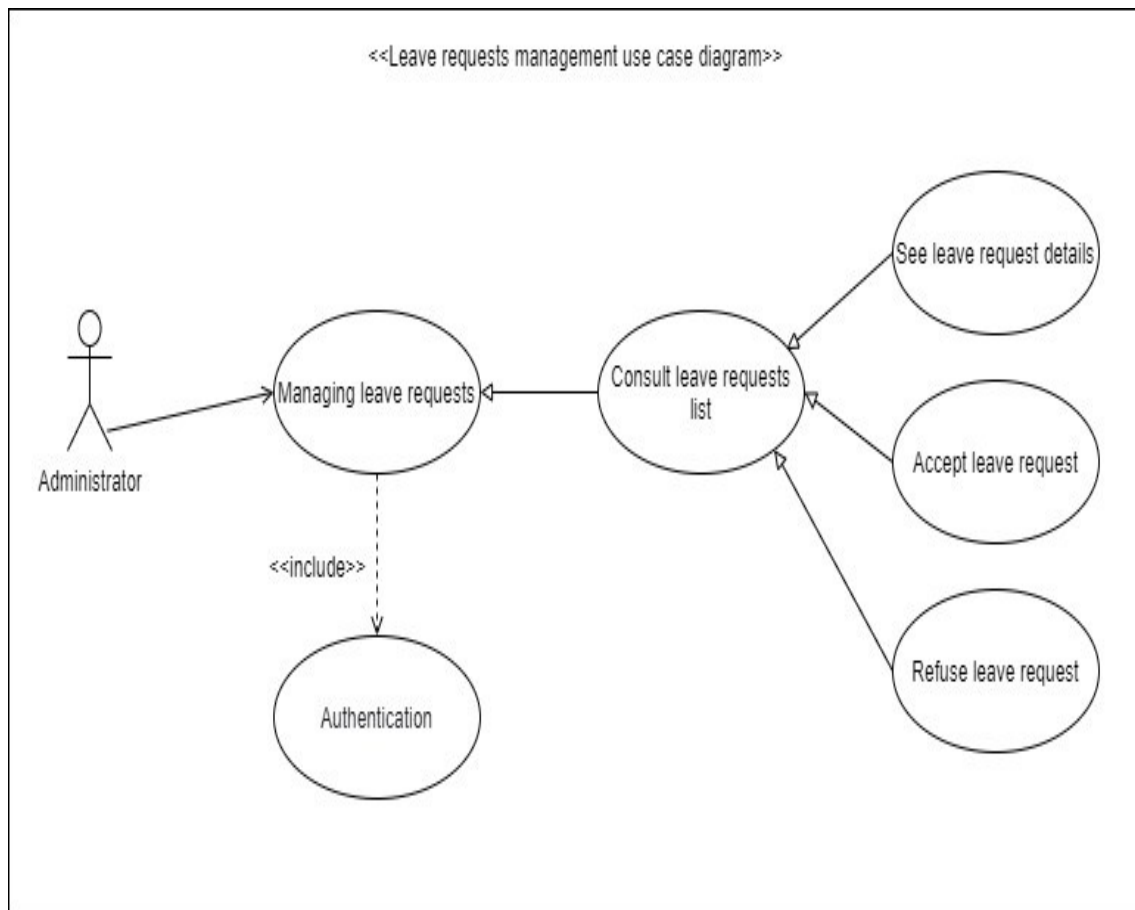


Figure 2.7: Use case diagram of “Leave requests management”

If he is authenticated, the administrator is able to manage leave requests by consulting a list of leave requests. This list gives him the possibility to see the details of the request and decide either to accept or refuse it.

Use case diagram of “Planning management”:

In the following diagram, we could observe the different action of the administrator to manage each of the merchandiser’s planning.

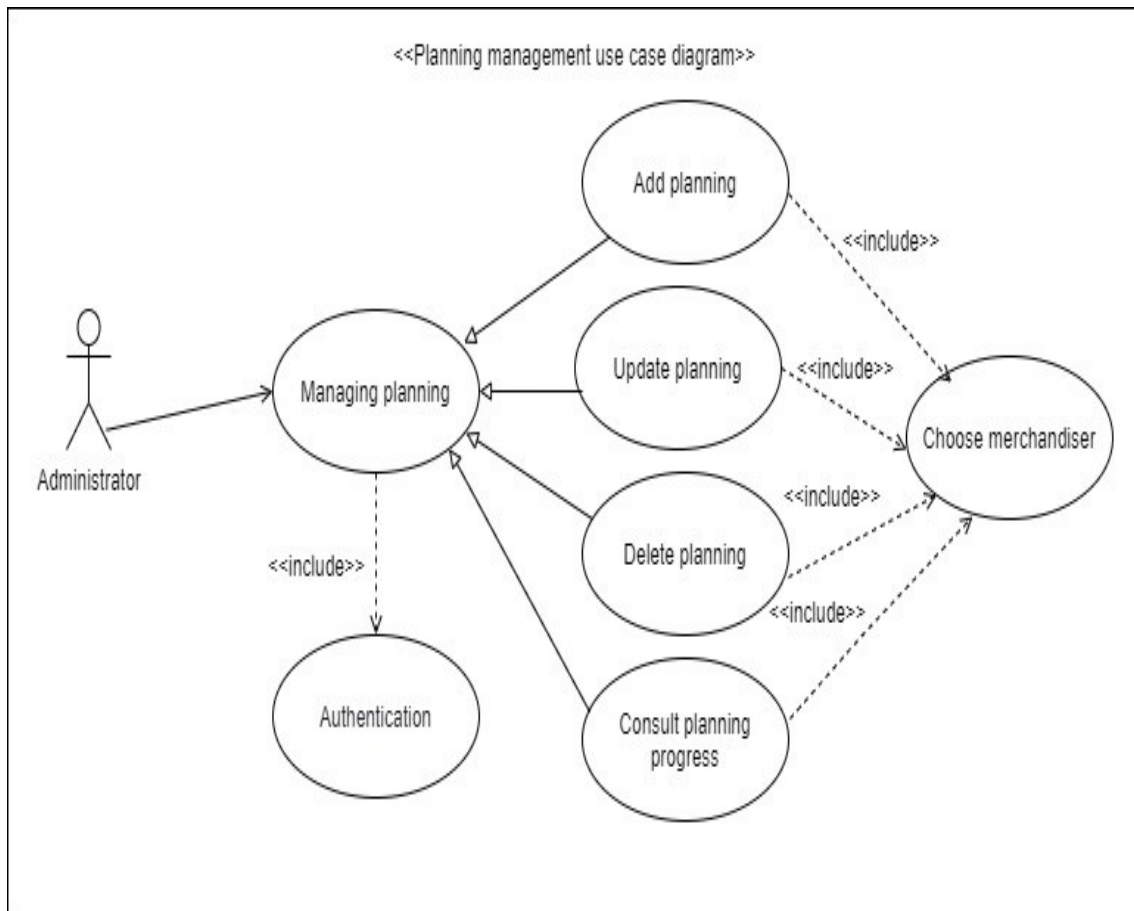


Figure 2.8: Use case diagram of “Planning management”

After authentication , the administrator is able to manage each of the merchandisers’ planning by either adding a planning where he would specify the start and end date of that planning, and a list of GMS the merchandiser have to visit per day, or he could update an already existing planning or even delete it.

He has the possibility to consult the progress of each merchandiser in his own planning too.

Use case diagram of “using the messaging service”:

This use case diagram is mainly about the actions of the administrator in order to use the messaging service utility.

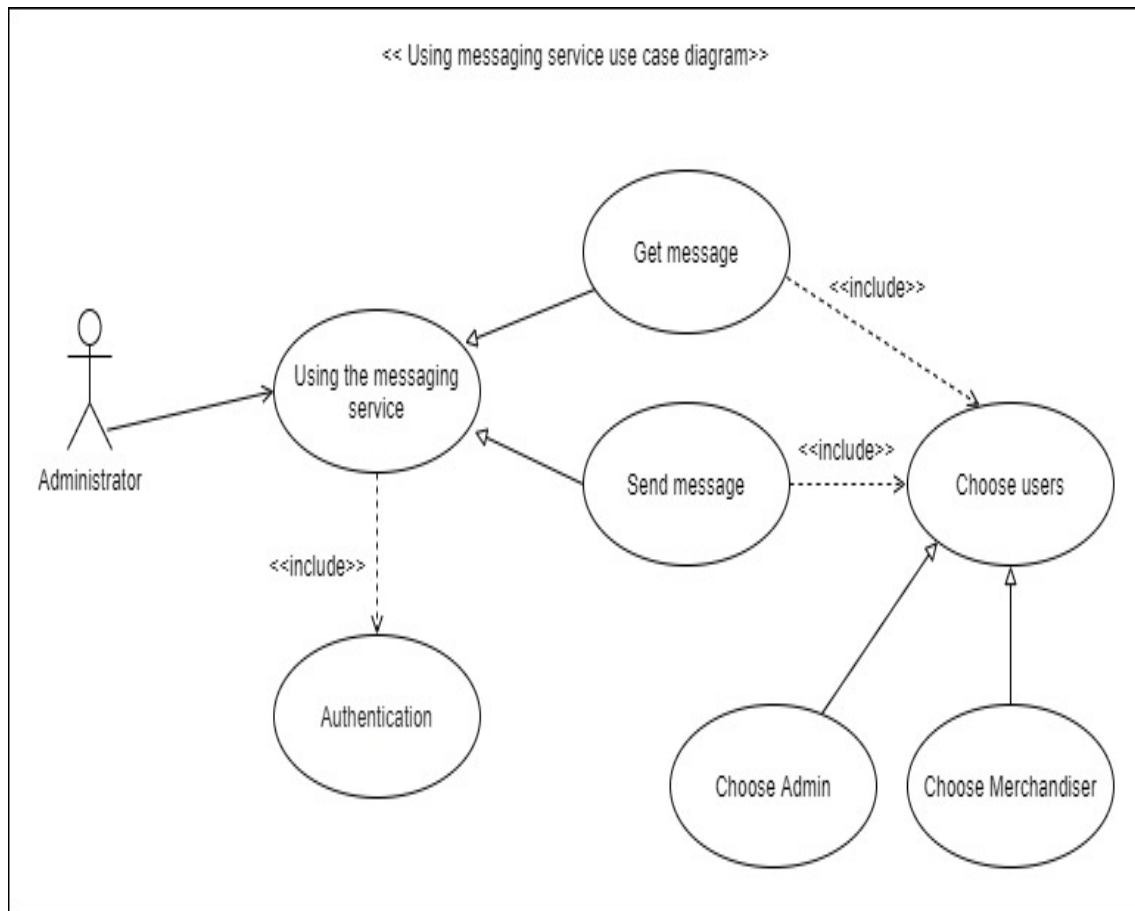


Figure 2.9: Use case diagram of “Planning management”

After authentication, the administrator is able to send messages to the merchandisers by choosing the receiver and getting messages back as well.

¹⁰conceptual study

Use case diagram of “Product category management”:

The following use case diagram illustrates the possible actions that could be done by the administrator to manage product categories.

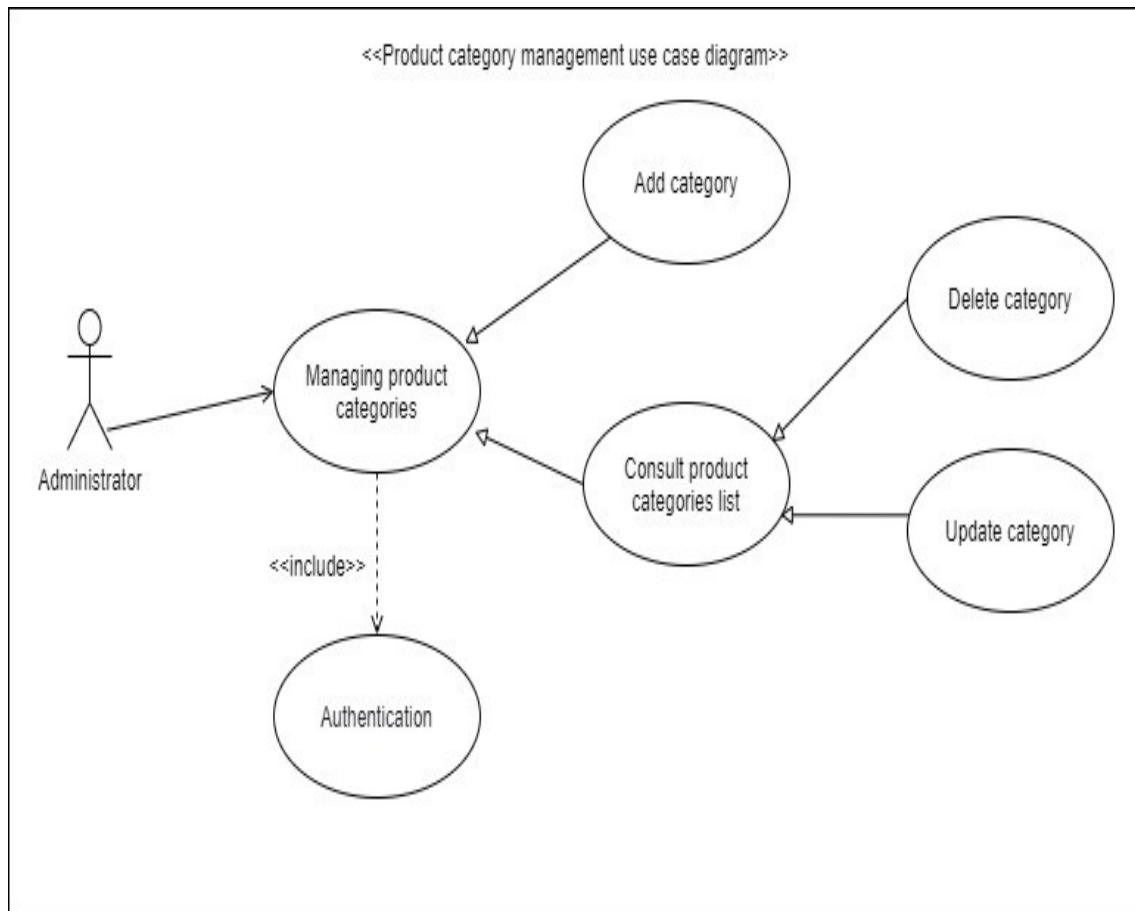


Figure 2.10: Use case diagram of “Product category management”

After authentication the administrator is able to manage product categories, he could add a new category or consult the product categories list in which he could either delete an existing category or update one.

¹¹conceptual study

Use case diagram of “Reclamation type management”:

The following use case diagram explains various possible action that could be done by the administrator in order to manage reclamation types.

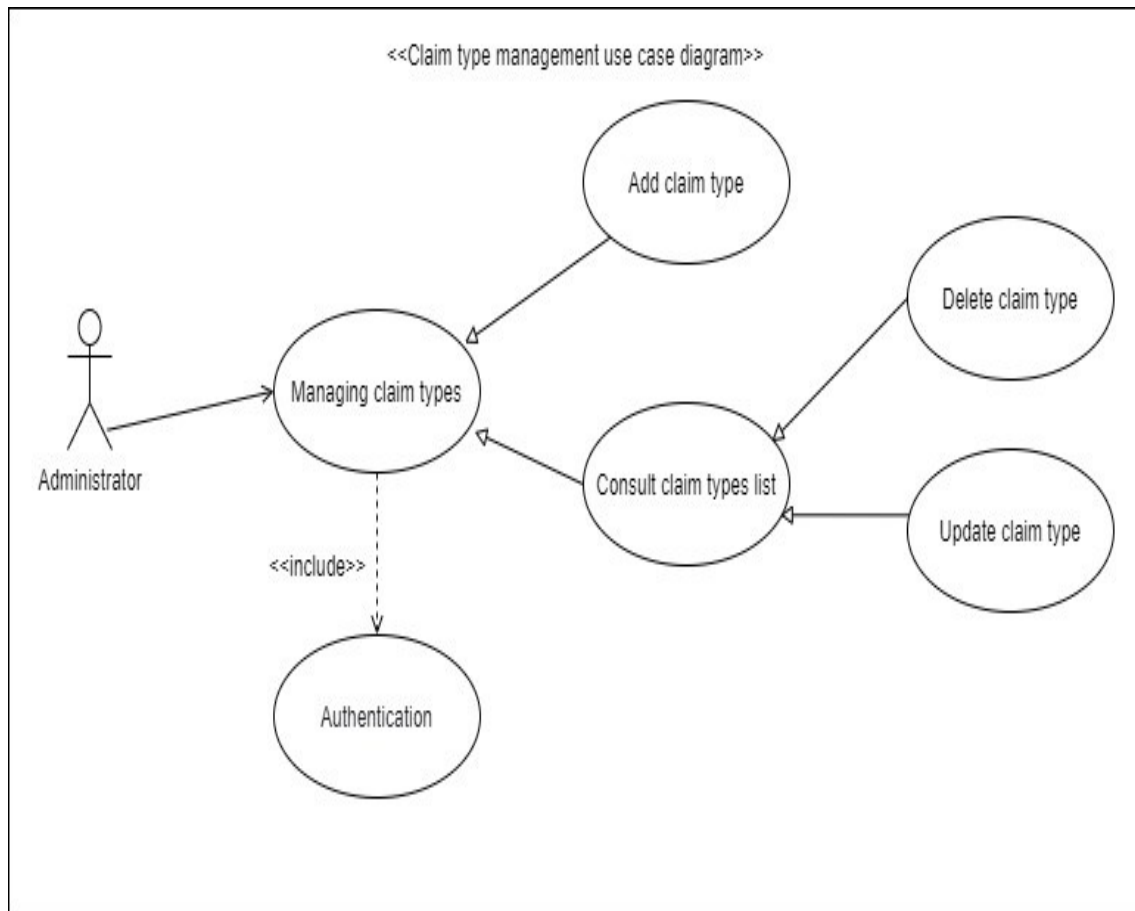


Figure 2.11: Use case diagram of “claim type management”

If he is authenticated, the administrator has the ability to manage claim types by adding new types, consulting the list of existing types then he could either delete a type or update one.

2.2.2 Use case diagram of the merchandiser

2.2.2.1 Global use case diagram

The figure below presents a global use case for the merchandiser in our application.

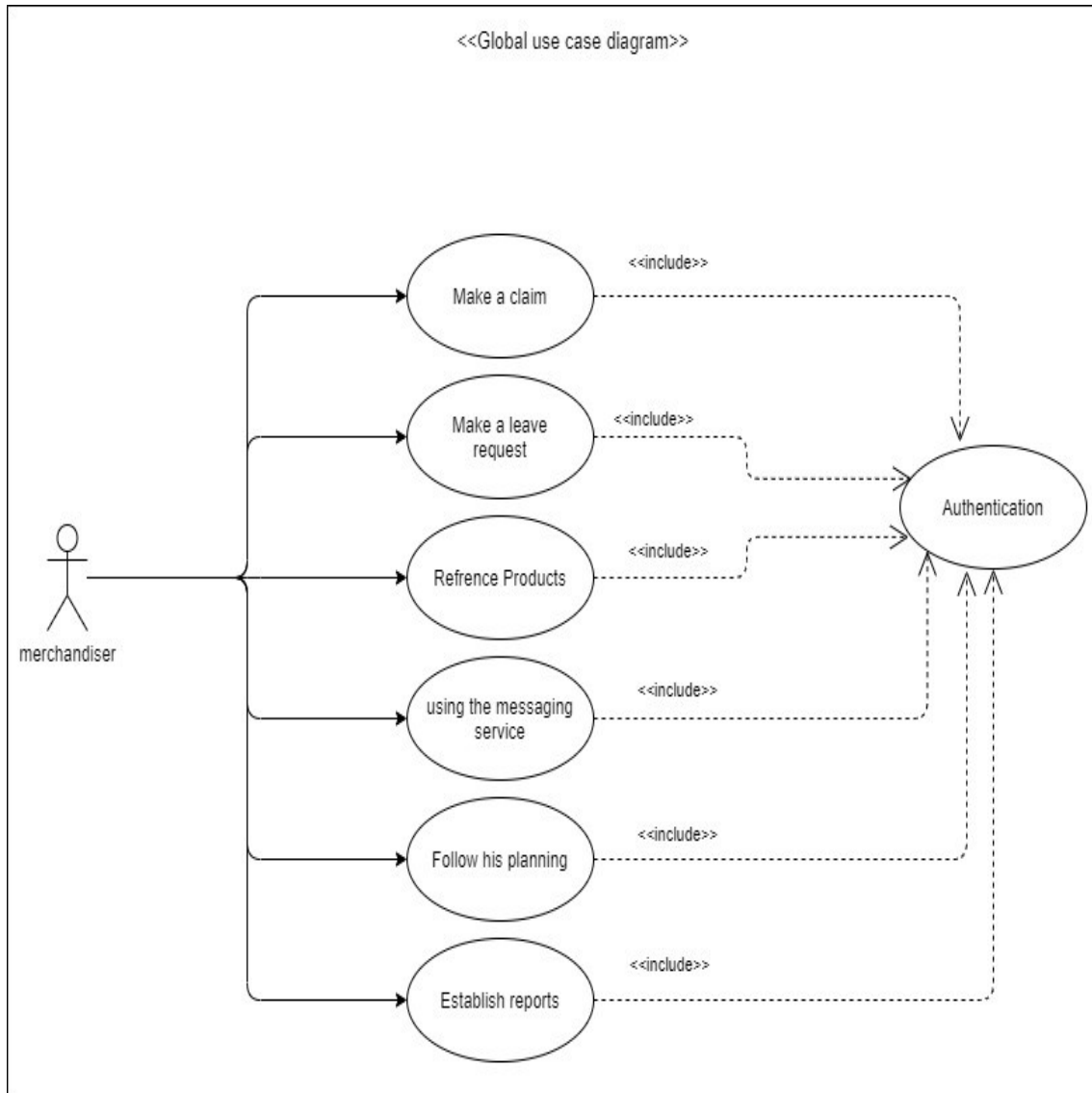


Figure 2.12: Use case diagram of the merchandiser

Use case diagram of “Establishing a report”:

The following use case diagram explains various possible action that could be done by the merchandiser in order to establish a report.

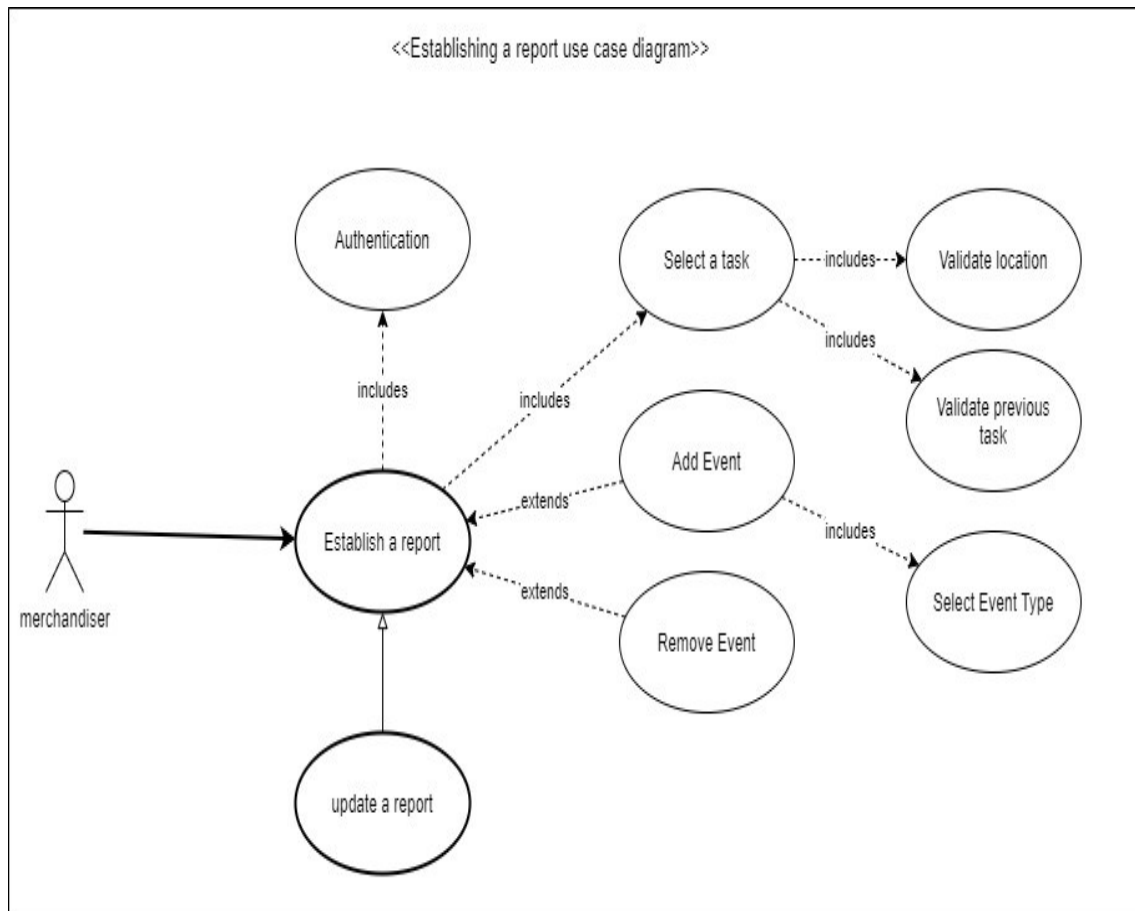


Figure 2.13: Use case diagram of “Establishing a report”

After authentication the merchandiser is able to establish a report by selecting a task, validating its location and verifying that the previous task was validated. The merchandiser could add a new event by selecting its type and he has the possibility to remove one or update its content.

Use case diagram of “Reference products”:

This use case diagram is mainly about the actions of the merchandiser in order to reference products.

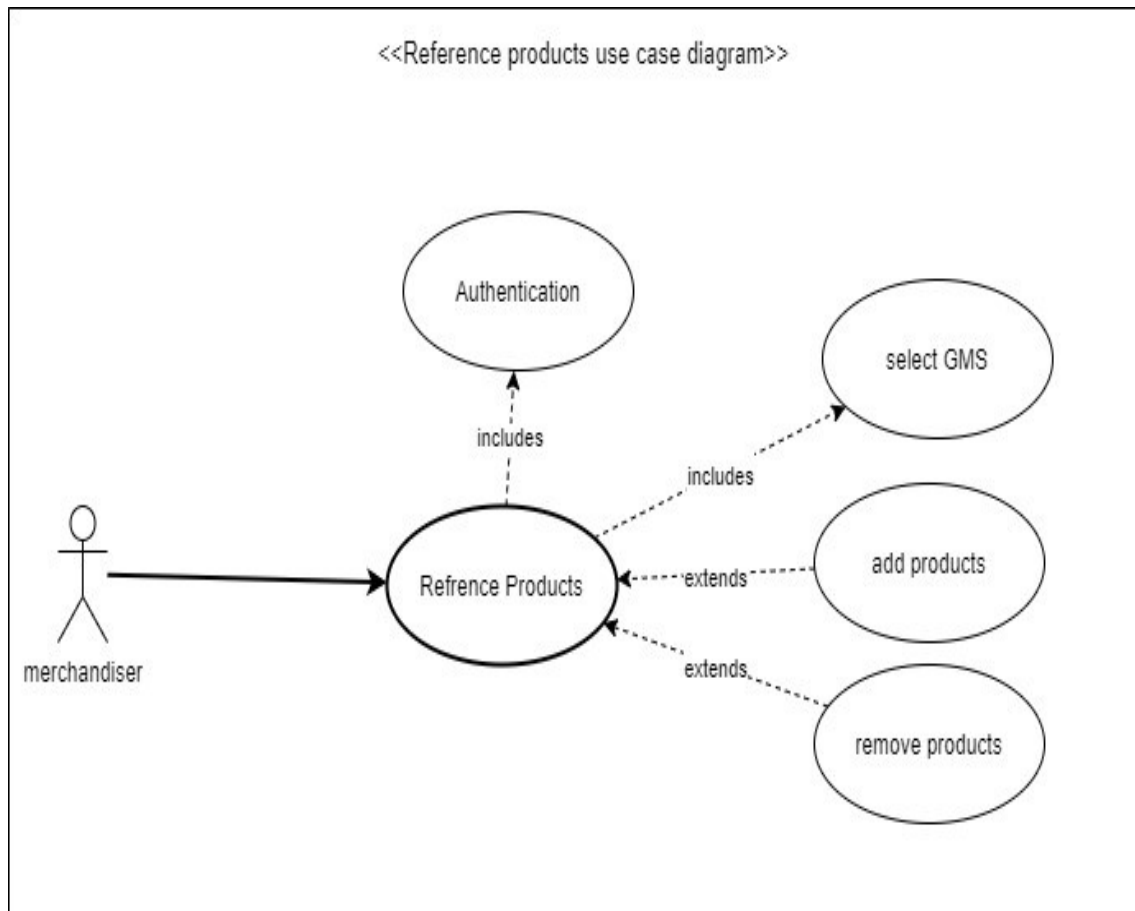


Figure 2.14: Use case diagram of “Establishing a report”

After authentication the merchandiser have the possibility to reference products.

To achieve that he has to choose the appropriate GMS and add or remove products to it.

Use case diagram of “Using the messaging service”:

In this use case diagram, we explain possible actions that could be done by the merchandiser to use the messaging service.

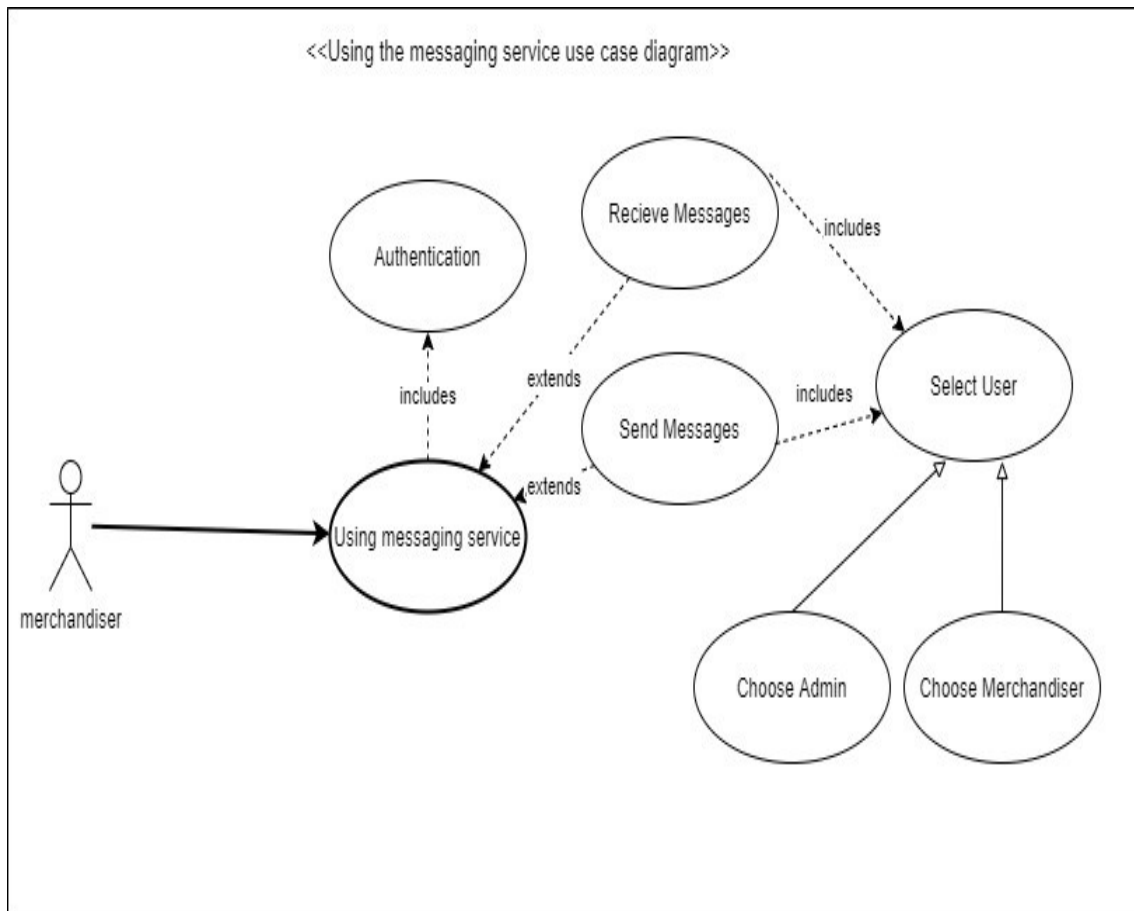


Figure 2.15: Use case diagram of “Using the messaging service”

After authentication the merchandiser has the possibility to use the messaging service by selecting a receiver that would allow him to send and receive messages or see the previous ones.

2.3 Class diagram

Class diagrams are the main building blocks of every object-oriented method. The class diagram can be used to show the classes, relationships, interface and association.[2]

(for clearer class diagram see[annex A])

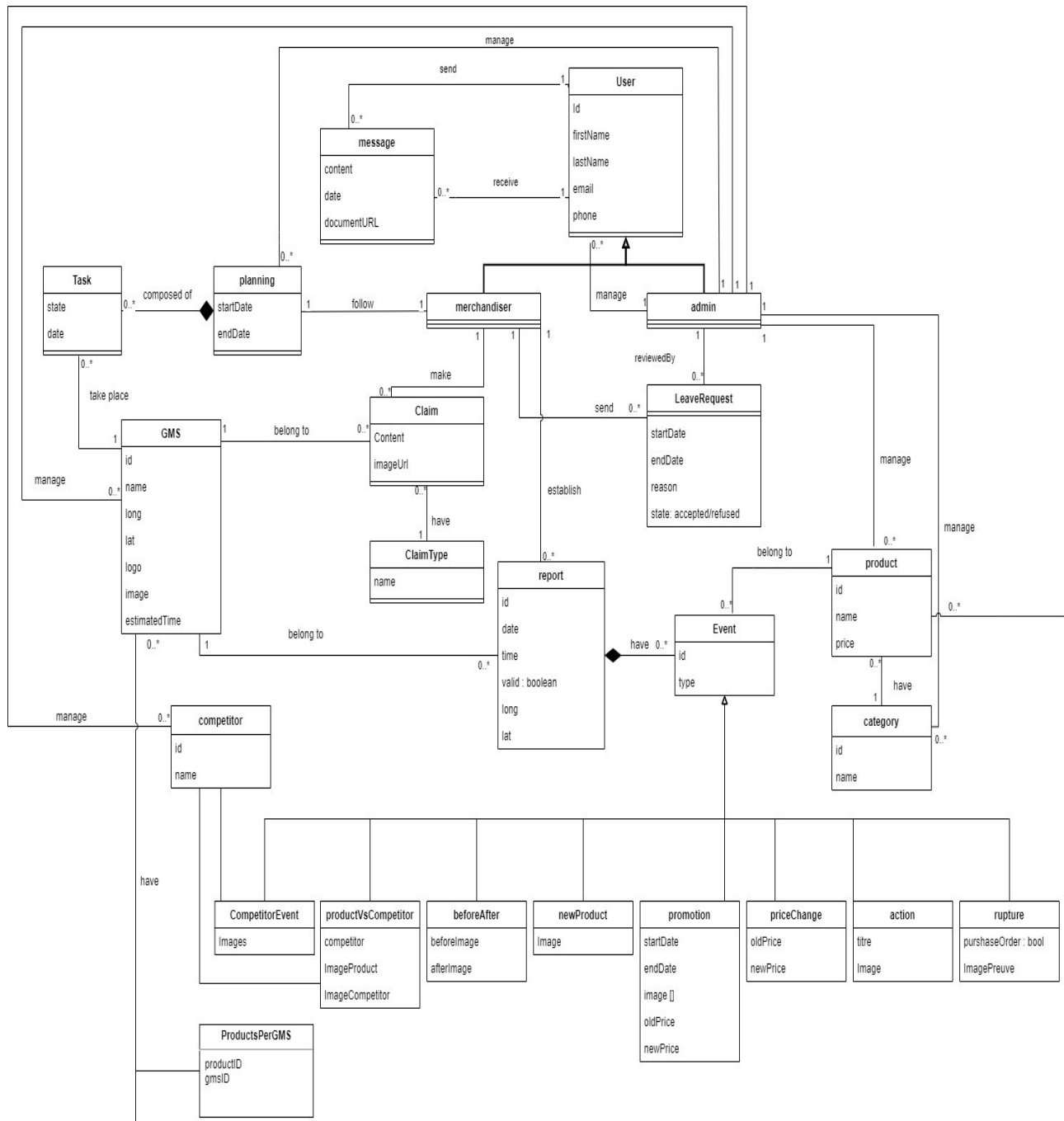


Figure 2.16: Class diagram

2.4 Sequence diagram

A sequence diagram simply depicts interaction between objects in a sequential order (the order in which these interactions take place). Sequence diagrams describe how and in what order the objects in a system function.[3]

2.4.1 Sequence diagram of “Login”

The following sequence diagram illustrates the sequence of interactions between objects in order to login.

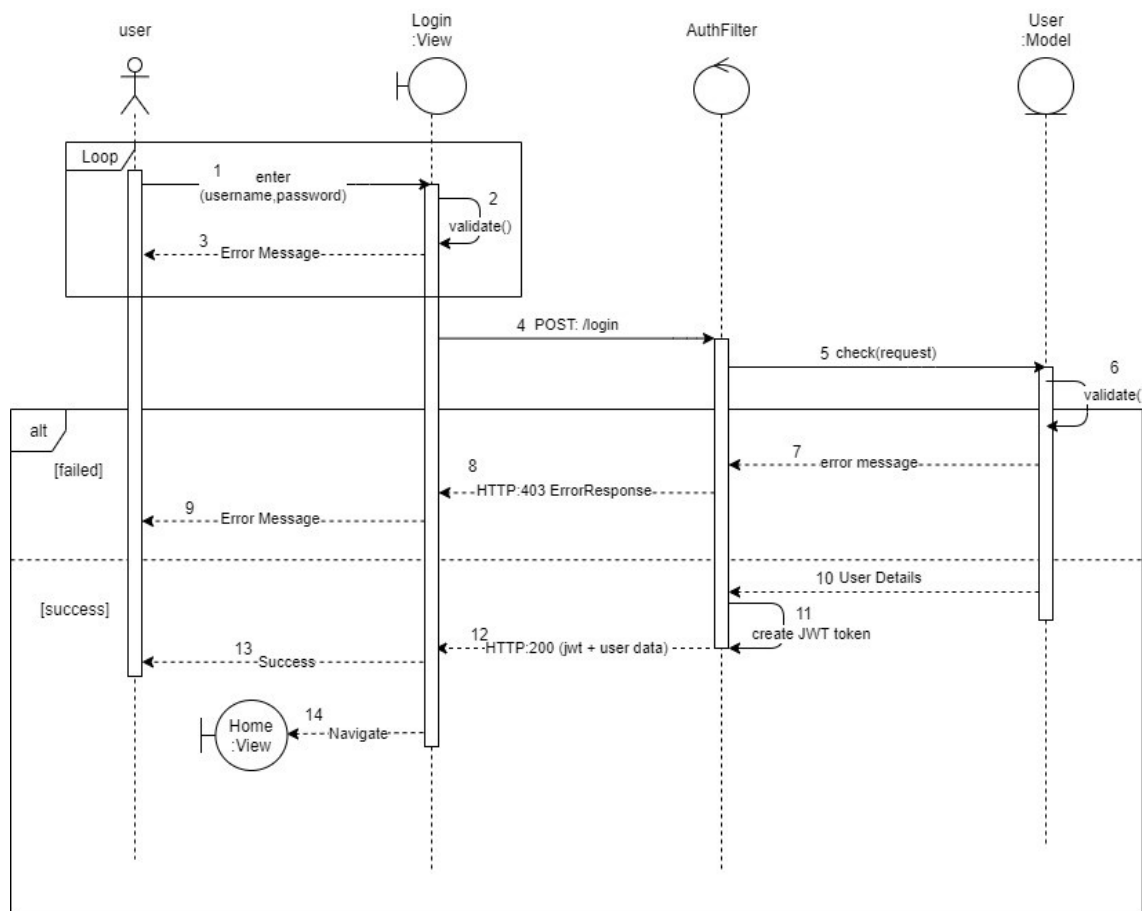


Figure 2.17: Sequence diagram of “Login”

In order to login, the user has to enter his username (which is the email in our case) and his password, a first validation in the frontend takes place to verify the format of the email.

¹⁸conceptual study

If the first validation is verified a post request is sent to the authentication filter of spring security which sends the request to user model in order to verify the existence of that particular user.

If a user with the specified email and password do not exists an error message is returned from the user model to the authentication filter which returns an error response to the view.

In case the specified user exists, user details are returned from the model to the authentication filter which creates a JWT token and returns it to the view alongside the user's data to signify the operation's success and the user is taken to the home page.

2.4.2 Sequence diagram of “create user”

The following sequence diagram illustrates the sequence of interactions between objects in order to create a leave request.

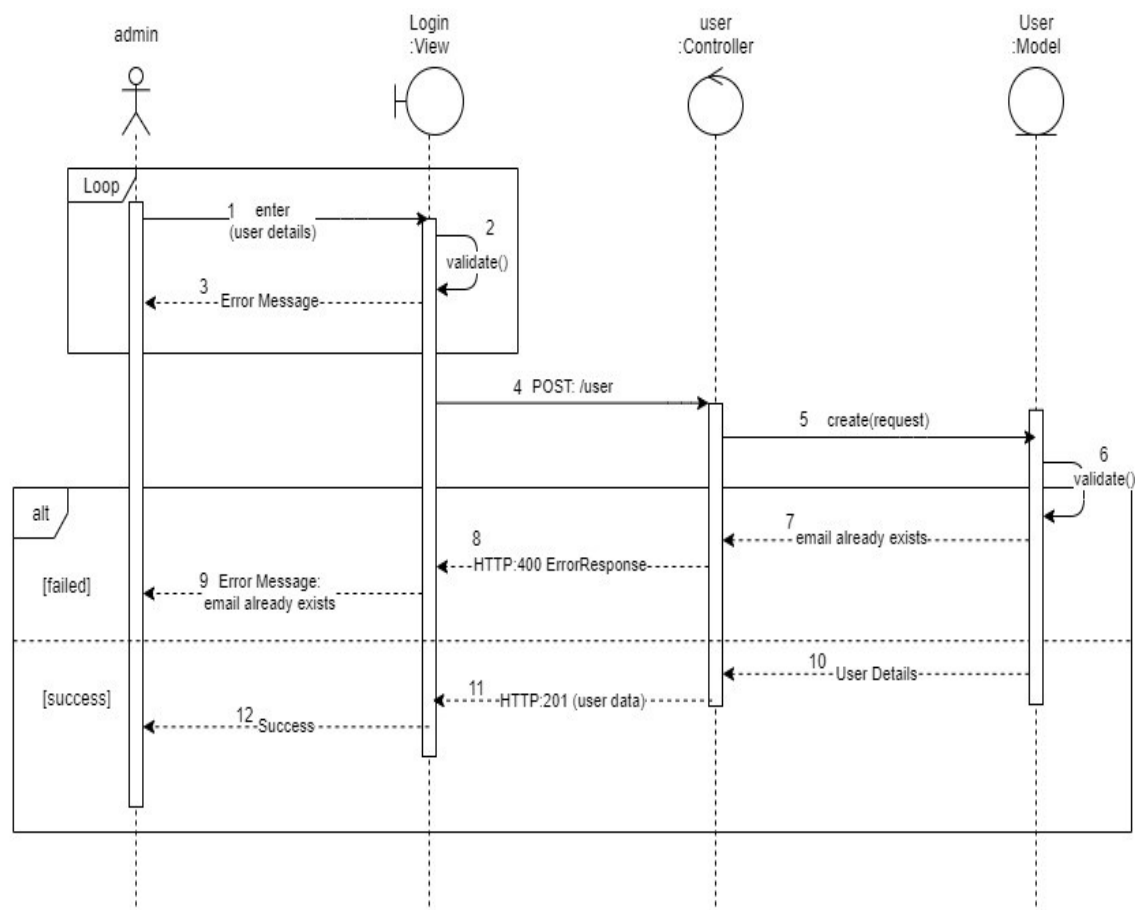


Figure 2.18: Sequence diagram of “create user”

To create a new user, the admin enters various user details in a form which is verified in the view to ensure all needed data where typed and display an error in case some fields were missing.

If all needed fields are filled a post request is send to the controller which created a request for the user model. The user model validate the existence of any other identical email (emails are our unique identifier in this application). In case another identical email exists in our database the model informs the controller which returns an error response in an http response to the view indicating that the typed email already exists.

In case no other identical email is found in our database, the new user is added and user details are returned to the controller which returns an Http response indicating the success of the operation to the user.

2.4.3 Sequence diagram of “create a leave request”

The following sequence diagram illustrates the sequence of interactions between objects in order to create a leave request.

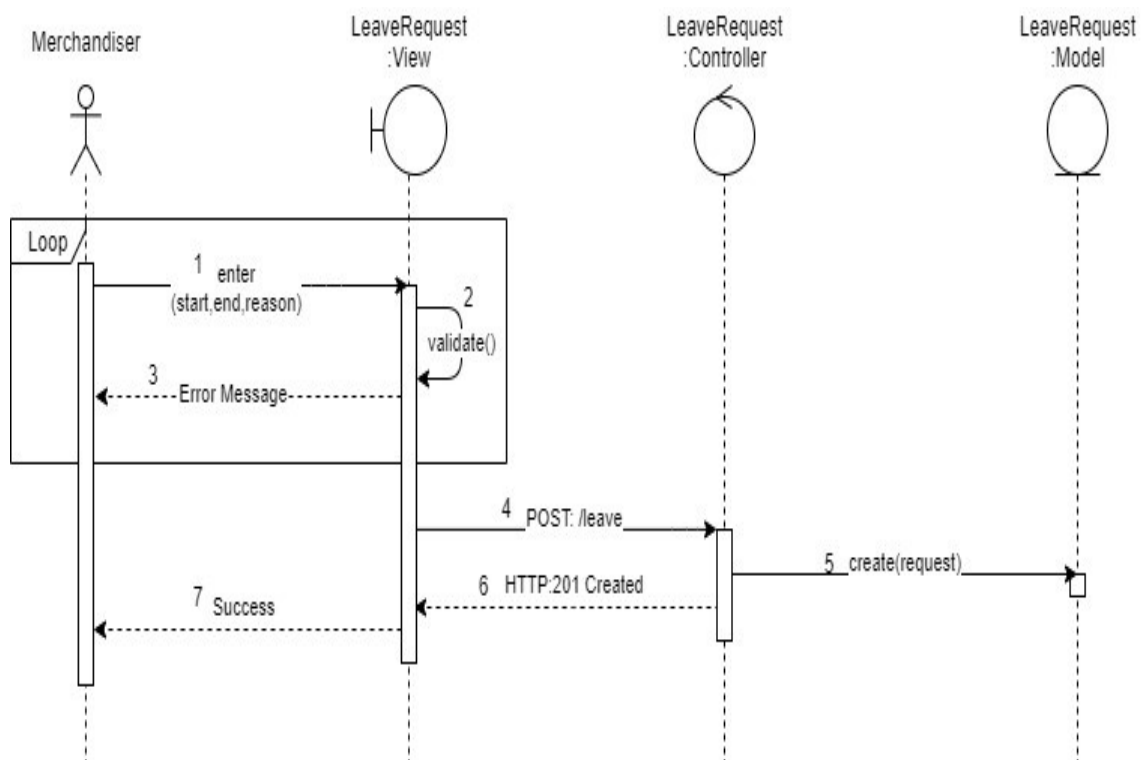


Figure 2.19: Sequence diagram of “create a leave request”

²⁰conceptual study

In order to create a leave request, the merchandiser has to specify the start date, end date and the reason for his request, a validation in the view takes place to ensure those data are specified and returns an error in case anything is missing. If all needed fields are filled an http post request is send the controller which in his turn sends a create request to the model and an http response to the view indicating the success of the operation.

Conclusion

This chapter was mainly focused on conception, a major cornerstone in software development. In the first part we defined our design methodology as UML, then we used it to create our use case diagrams representing various interactions between different actors and use cases.

After that, we represented our sequence diagrams showing sequential interactions between the objects of our system to finish with a class diagram defining the global structure of our solution in terms of classes and relationships.

Once the system was clarified in terms of behavior, interactions and structure we move forward to realization which we treat in the next chapter.

²¹conceptual study

CHAPTER 3

REALISATION

Introduction

In this last chapter we focus on realization, the final step in software development. This is the phase where we code our application.

First of all we start by defining our global architecture and our choice of technologies to finish by presenting an overview of our solution through screenshots

3.1 System architecture

Prior to delving into our choice of technologies, it is beneficial to take a look at the architecture that compose our system.

3.1.1 N-tier architecture

N-tier architecture or multi-layer architecture is client-server architecture that separates responsibilities between the layers. Usually we have 3 tiers which handle presentation, business logic, and data respectfully.

¹Réalisation

This separation of concerns allows the developer to create scalable and highly maintainable software.

- **Presentation Tier** : this is the client or the user interface , it is the layer that is presented to the end user.
- **Business Logic Tier** : where we find most of the logic for a system, usually as a middleman between what the user sees and the stored data.
- **Data Tier** : the place where all the data used by the application is stored (commonly a database)

The following is a diagram of the representation of the 3 tiers in our application :

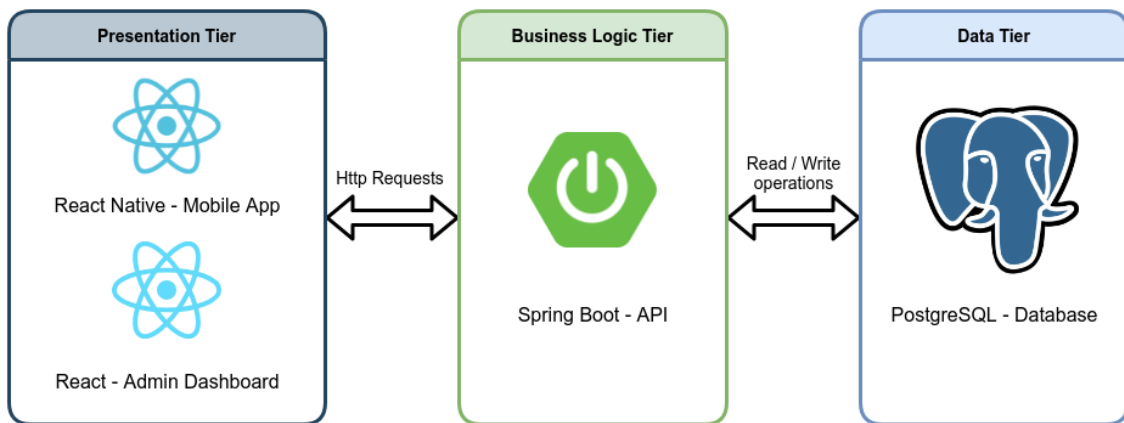


Figure 3.1: Tier architecture

3.1.2 MVC pattern

Model-View-Controller or MVC for short is a software design pattern widely used for developing user interfaces. This pattern facilitates the development of systems where we need to separate the logic of the way data is presented to and accepted from the user. MVC is very popular in the world of web development and therefore for each popular language there exists a MVC framework that helps in the implementation of the pattern.

²Réalisation

A general way to describe MVC is the following :

- **View** : In accordance with the name this is what the user sees, in the case of a mobile app that refers to the different screens.
- **Model** : A dynamic data structure that represents stored data in the app (for example each model could represent a database table)
- **Controller** : The main piece of the system, generally handles most of the logic and ensures communication between the view and the model.

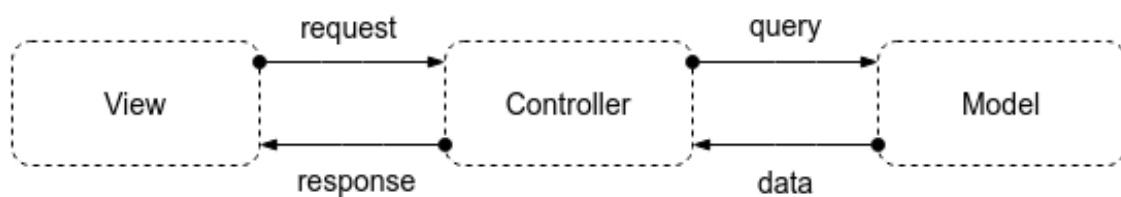


Figure 3.2: Interaction within MVC

3.2 Technology choices

3.2.1 Spring Boot



Figure 3.3: Spring Boot

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run" with minimal Spring configuration.[4]

Spring Boot has various advantages as it allows to create a standalone spring applications with embedded Tomcat as a web server. It provides starter dependencies which minimize manual configurations to the lowest.

³Réalisation

3.2.2 React



Figure 3.4: React

React is a declarative, component based JavaScript library for building user interfaces.[5] Created and supported by Facebook, React is one of the most popular libraries for web development. React is lightweight, based on flexible, reusable components and a rich third party libraries environment ensuring a fast development process and a good UI quality.

3.2.3 Database



Figure 3.5: PostgreSQL

PostgreSQL is a powerful, open source object-relational database system with over 30 years of active development that has earned it a strong reputation for reliability, feature robustness, and performance.

PostgreSQL comes with many features aimed to help developers build applications, administrators to protect data integrity and build fault-tolerant environments, and help you manage your data no matter how big or small the dataset. In addition to being free and open source, PostgreSQL is highly extensible.[6]

⁴Réalisation

3.3 Development environment and tools

In this part we present different IDEs and tools used to build this application.

3.3.1 Visual Studio Code



Figure 3.6: Visual Studio Code

Visual Studio Code combines the simplicity of a source code editor with powerful developer tooling, such as code completion, debugging and offers a vast choice of free community driven extensions. Visual Studio Code supports macOS, Linux, and Windows.

Visual Studio Code features a lightning fast source code editor, perfect for day-to-day use. With support for hundreds of languages, VS Code helps you be instantly productive with syntax highlighting, bracket-matching, auto-indentation, box-selection, snippets, and more.[7]

3.3.2 Spring Tool Suite



Figure 3.7: Spring Tool Suite

STS is an Eclipse-based development environment that is customized for the development of Spring applications. It provides a ready-to-use environment to implement, debug, run and deploy your applications. STS is built as an addition on top of the latest Eclipse releases.[8]

⁵Réalisation

3.3.3 Postman



Figure 3.8: Postman

Postman is a scalable API testing tool .It started in 2012 as a side project by Abhinav Asthana to simplify API workflow in testing and development. It offers a sleek user interface with which to make HTTP requests, without the hassle of writing a bunch of code just to test an API's functionality.

It is used by over 5 million developers every month to make their API development easy and simple. It has the ability to make various types of HTTP requests (GET, POST, PUT, PATCH), saving environments for later use.[9]

3.3.4 GitLab



Figure 3.9: GitLab

GitLab is an open source code repository and collaborative software development platform. It offers a location for online code storage and capabilities for issue tracking. The repository enables hosting different development chains and versions, and allows users to inspect previous code and roll back to it in the event of unforeseen problems.[10]

⁶Réalisation

3.3.5 Draw.io



Figure 3.10: Draw.io

draw.io is proprietary software for making diagrams and charts. The software allows you to choose from an automatic layout function, or create a custom layout. They have a large selection of shapes and hundreds of visual elements to make your diagram or chart one-of-a-kind.[11]

3.3.6 Beekeeper Studio



Figure 3.11: Beekeeper Studio

It is a free and open source SQL editor and database manager. It is cross-platform, and available for Linux, Mac, and Windows. Use it to query and manage your relational databases, like MySQL, Postgres, SQLite, and SQL Server.[12]

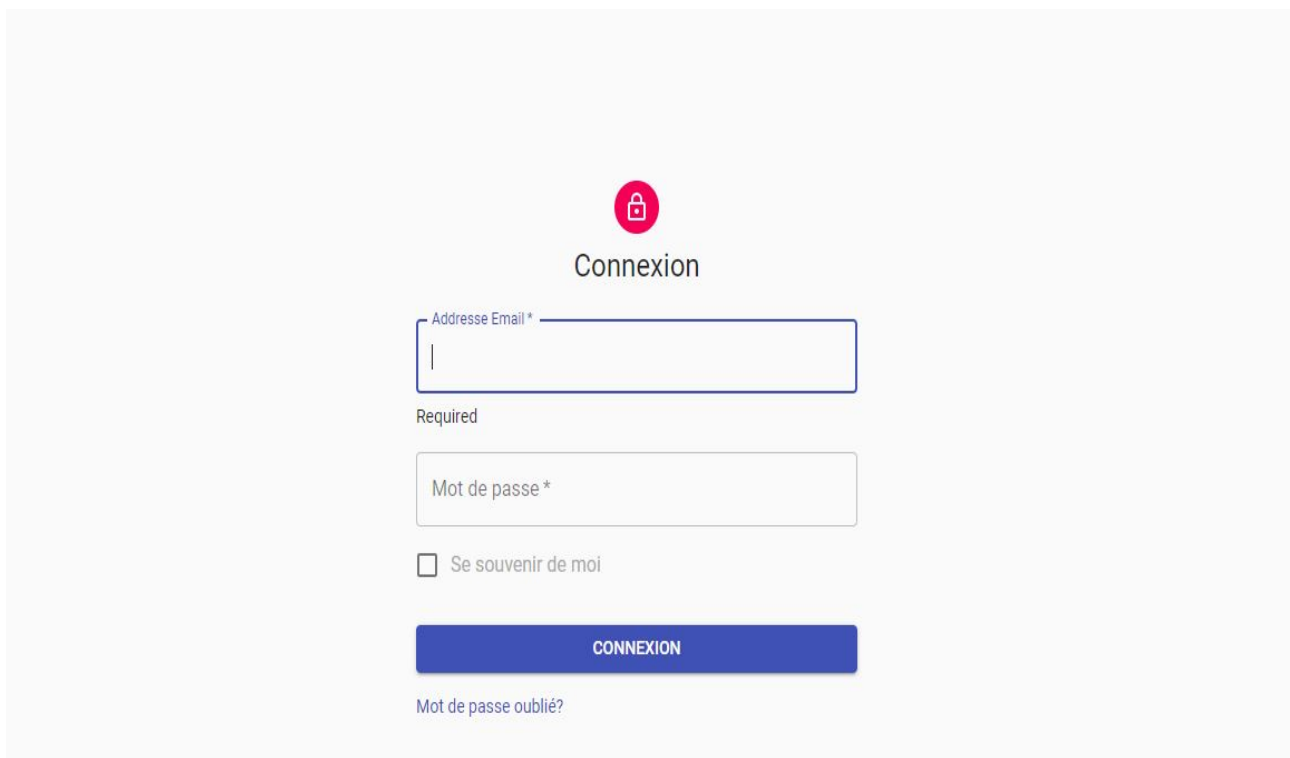
⁷Réalisation

3.4 Interfaces

3.4.1 Administrator

3.4.1.1 Login

In order to use the application the administrator has to log in from the interface shown in the figure <Figure 3.12>. He has to specify the email and password and then validate by clicking the connection button



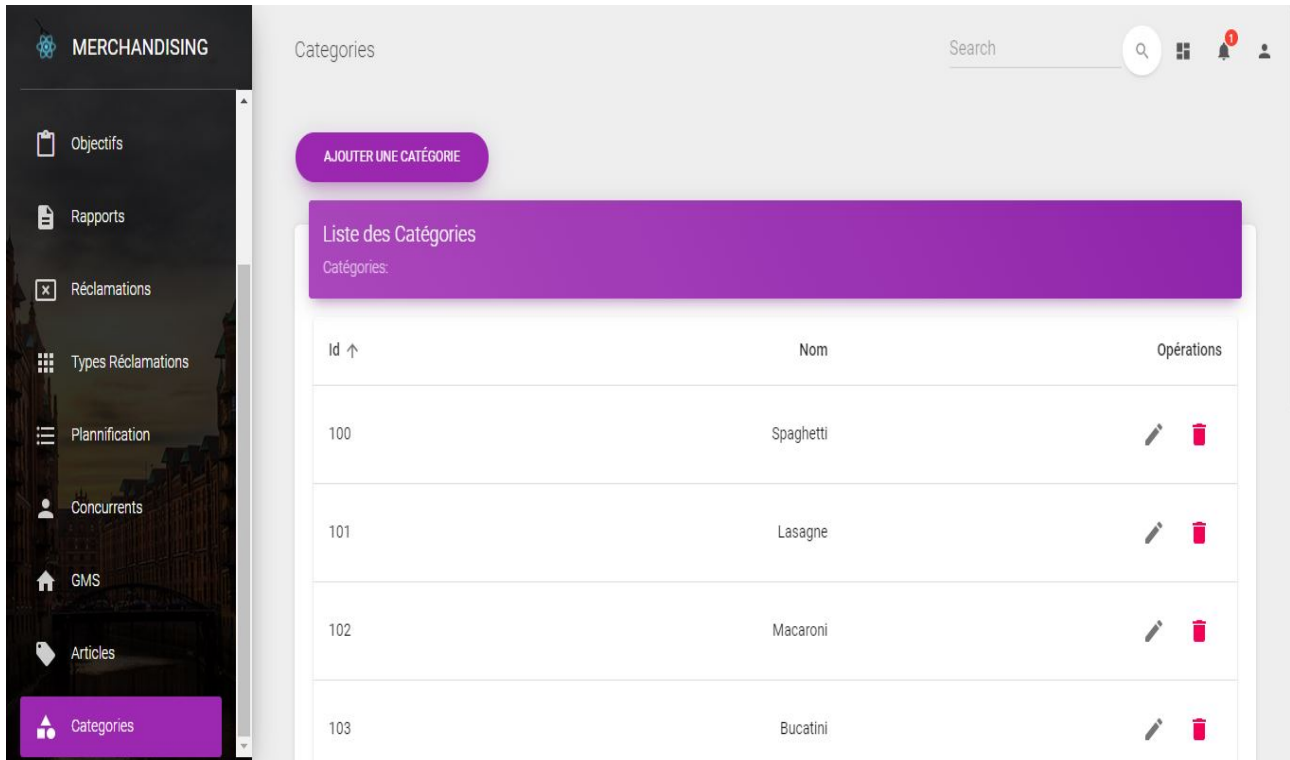
The image displays a login interface titled "Connexion". At the top center is a red circular icon containing a white padlock. Below the icon, the word "Connexion" is written in a black sans-serif font. The form consists of two input fields: the first is labeled "Adresse Email *" and contains a single vertical line cursor; the second is labeled "Mot de passe *". Below the email field, the word "Required" is written in a small, light gray font. Below the password field, there is a checkbox followed by the text "Se souvenir de moi". At the bottom of the form is a solid blue rectangular button with the word "CONNEXION" in white, uppercase letters. Below the button, the text "Mot de passe oublié?" is displayed in a small, light blue font.

Figure 3.12: Interface login

3.4.1.2 Categories management

See the list of categories:

By clicking on Categories in the sidebar, the list of all available categories is displayed in a table figure<Figure 3.13>



Id ↑	Nom	Opérations
100	Spaghetti	 
101	Lasagne	 
102	Macaroni	 
103	Bucatini	 

Figure 3.13: Interface lthe list of categories

Add a new category:

The administrator has the possibility to add a new category by clicking on the button “Add new category”. Once this is done a modal shows up with a form to specify the category’s name as shown in the figure <Figure 3.14>

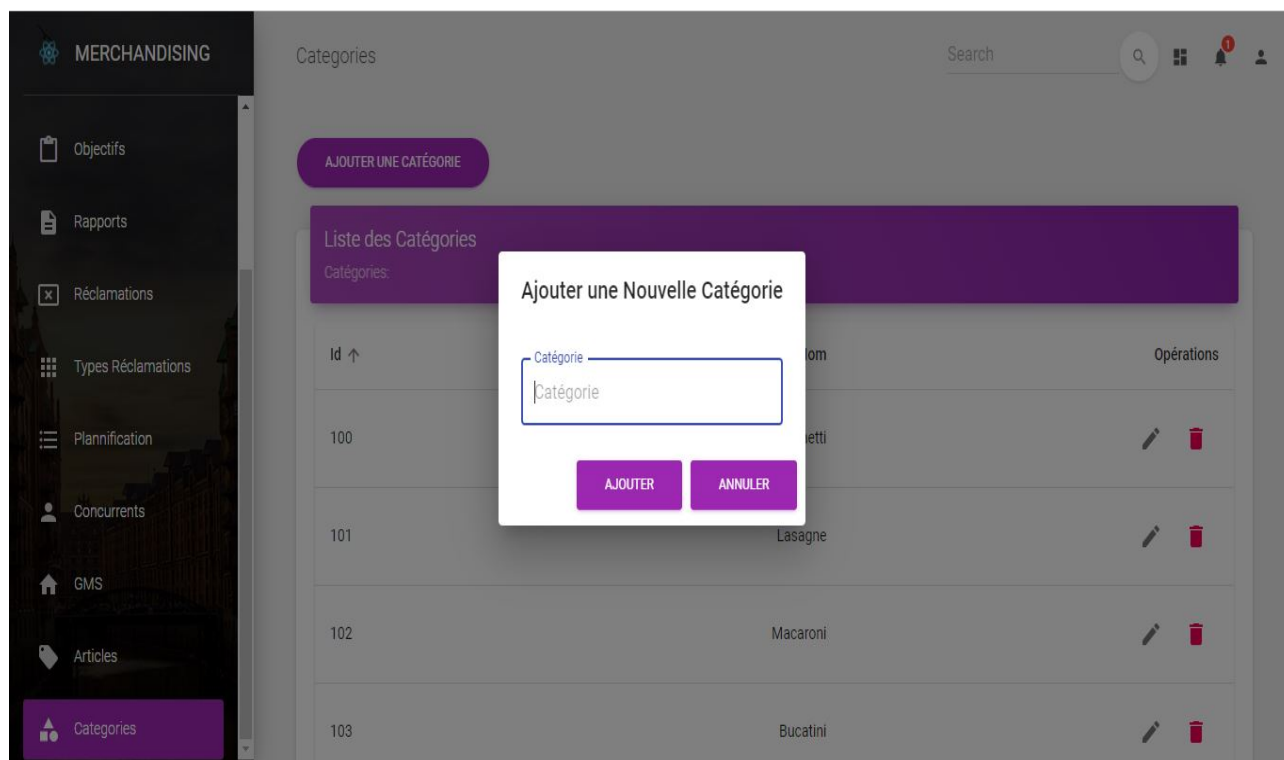


Figure 3.14: Interface Add a new category

Update a category:

Any category could be updated by clicking on the update icon in the operations section inside the table , Once that icon is clicked a modal is shown with the old category name allowing the administrator to modify it , the modal is shown in figure <Figure 3.15>

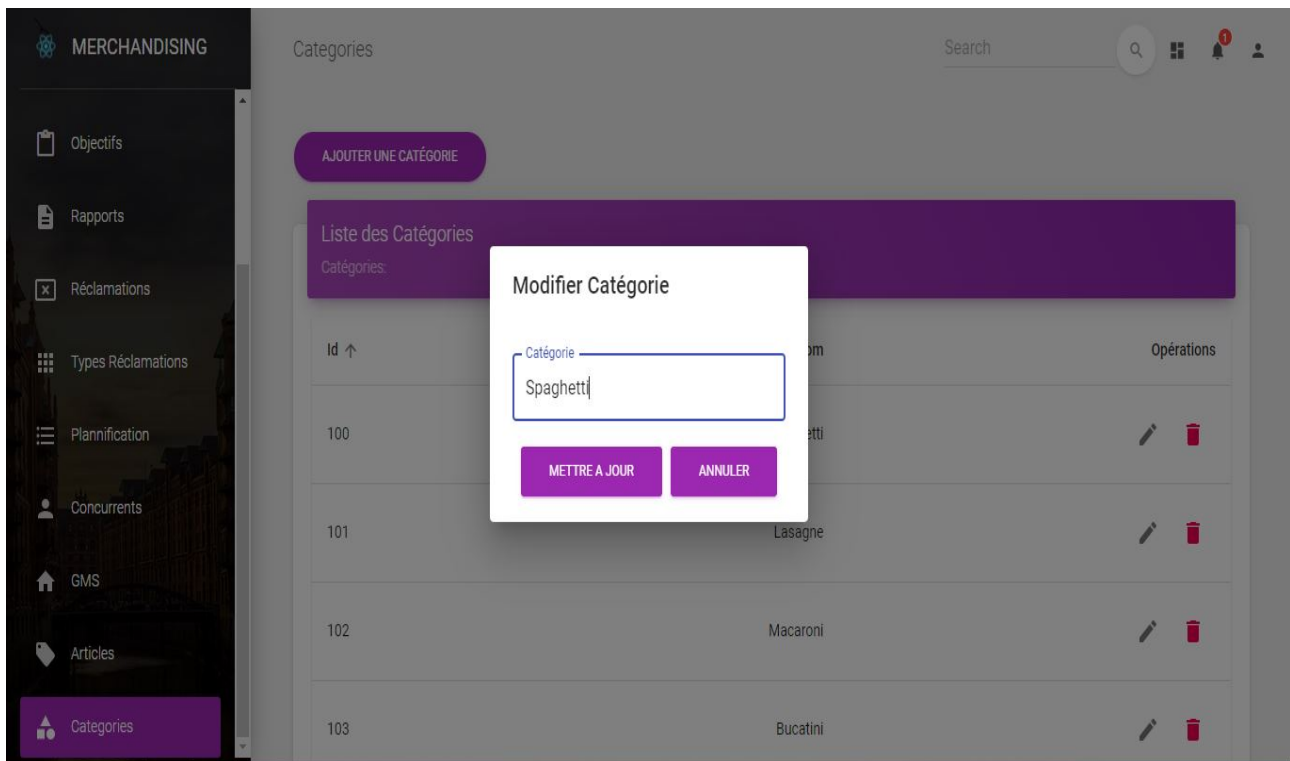


Figure 3.15: Interface Update a category

Delete a category:

The administrator could delete any category by clicking the delete icon in the operations section, the modal shown in figure<Figure 3.16> is displayed once that icon is clicked.

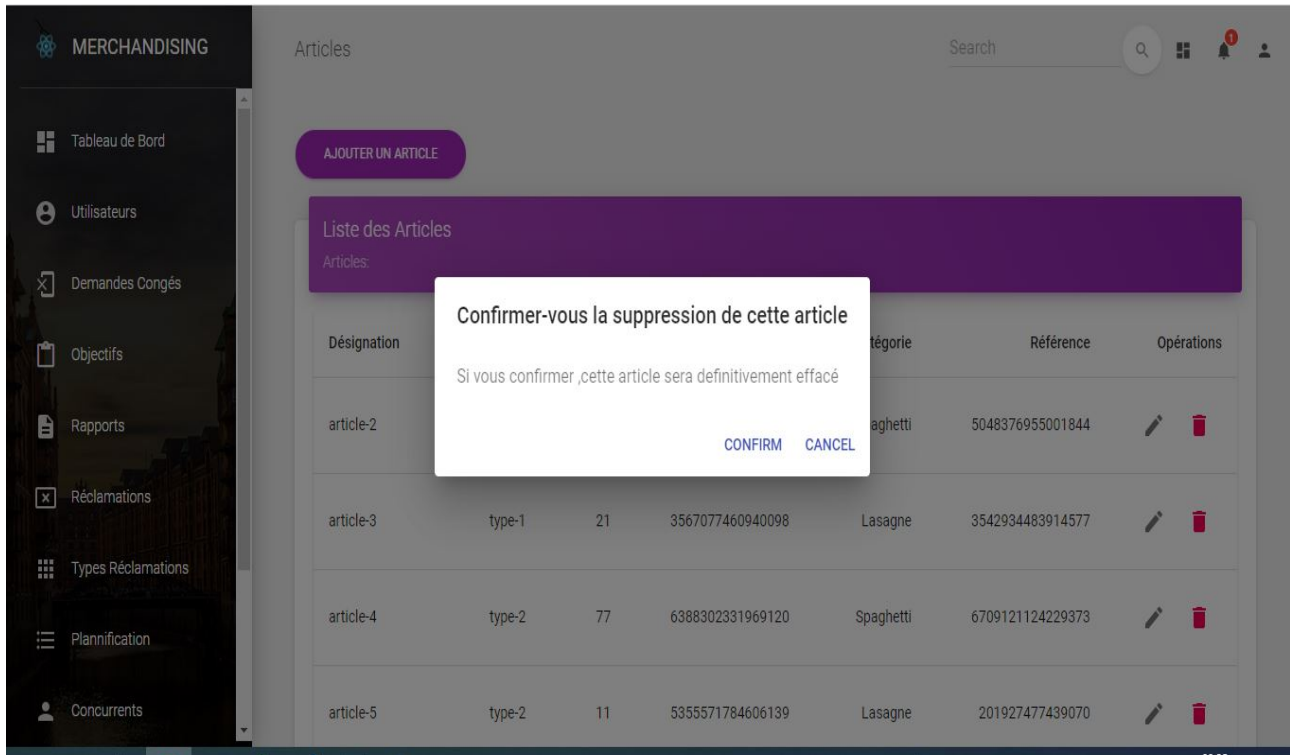


Figure 3.16: Interface Delete a category

3.4.1.3 Planning management

See a merchandiser's planning and monitor its advancement:

By choosing a merchandiser's name his planning is displayed in the calendar .Completed tasks are highlighted in a green color and other tasks in bleu as shown in figure«Figure 3.17»

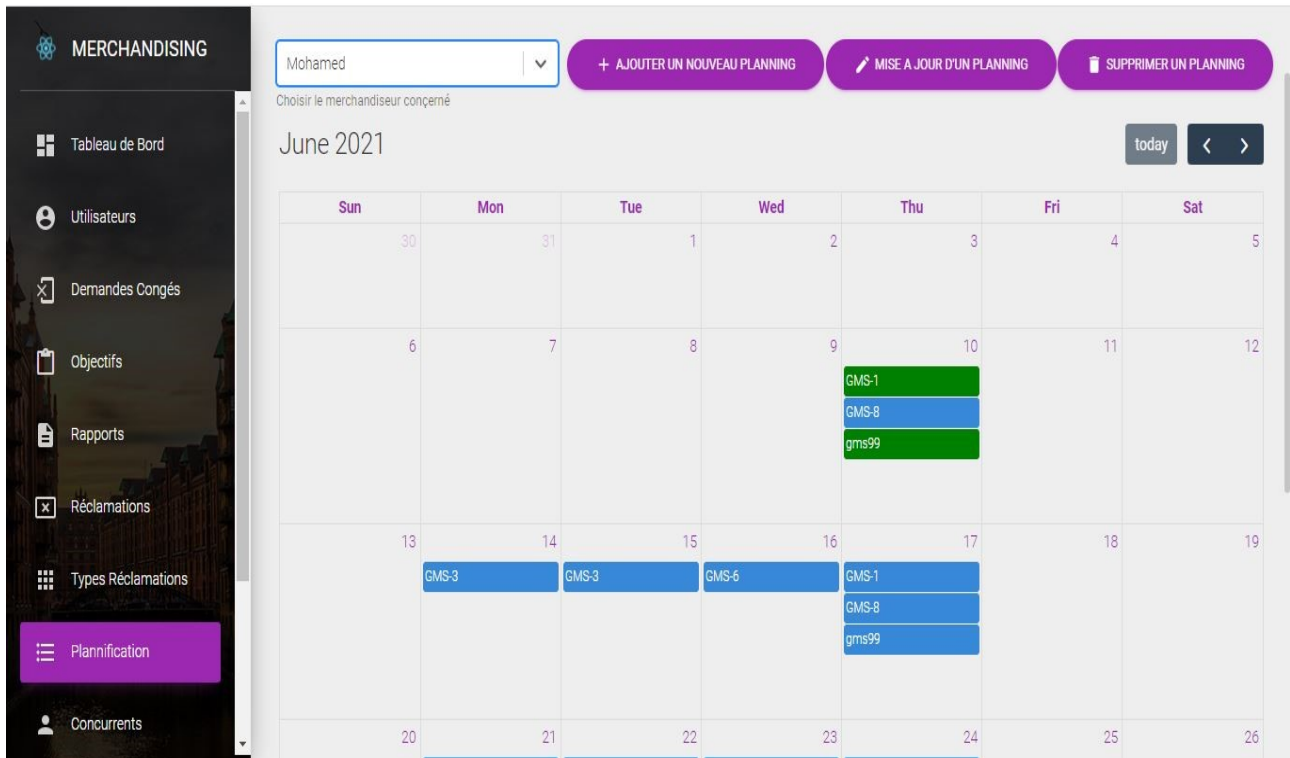
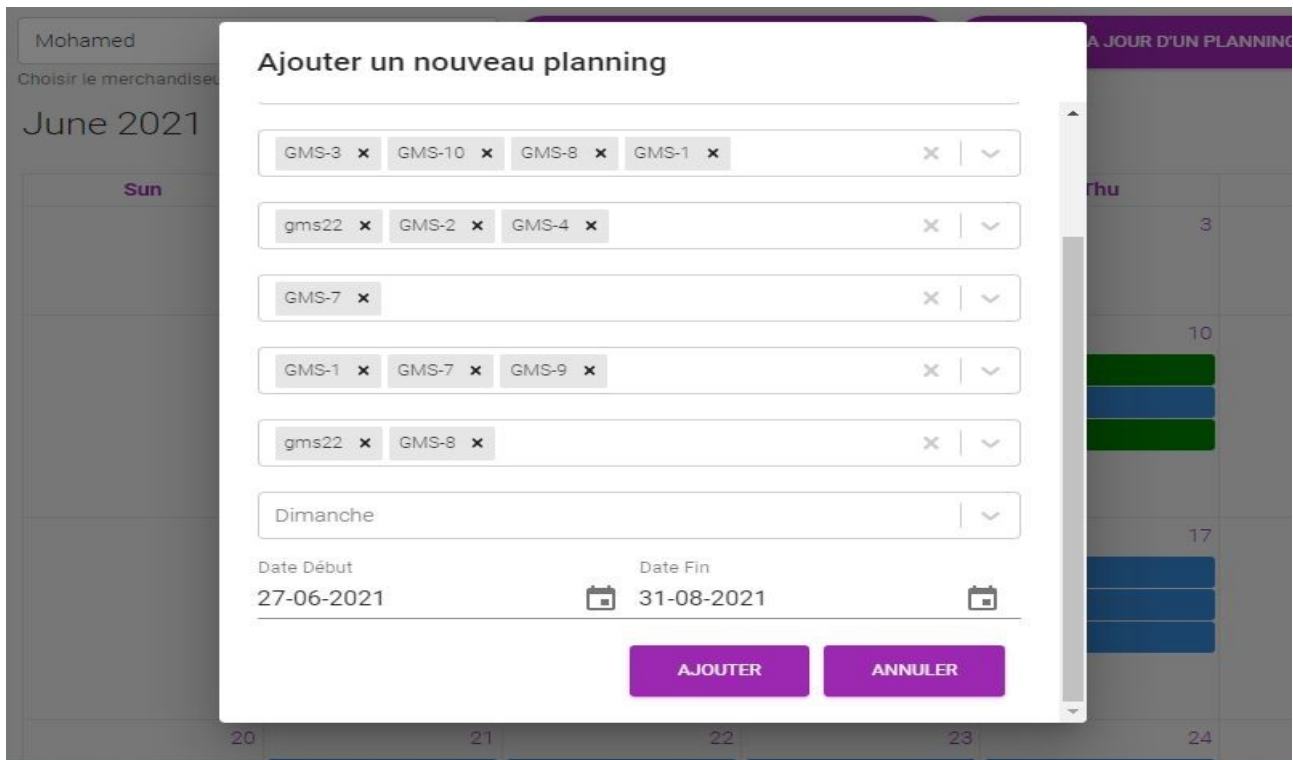


Figure 3.17: Interface merchandiser's planning and advancement

Add a new planning:

The administrator has the possibility to add a new planning by clicking on the button “Add new planning”. Once this is done a modal shows up with a form to specify the planning details as shown in the figure <Figure 3.18>



The image shows a web application interface with a modal window titled "Ajouter un nouveau planning". The modal is overlaid on a background that appears to be a calendar or planning tool. The modal contains several input fields and buttons:

- A header bar with the title "Ajouter un nouveau planning".
- Five rows of input fields, each containing a list of items with an "x" icon to remove them and a dropdown arrow to the right. The items are: "GMS-3", "GMS-10", "GMS-8", "GMS-1", "gms22", "GMS-2", "GMS-4", "GMS-7", "GMS-1", "GMS-7", "GMS-9", "gms22", "GMS-8", and "Dimanche".
- Two date fields: "Date Début" (27-06-2021) and "Date Fin" (31-08-2021), each with a calendar icon.
- Two buttons at the bottom: "AJOUTER" (Add) and "ANNULER" (Cancel).

The background interface shows a calendar for June 2021, with a sidebar on the left and a main area on the right. The sidebar has a search bar and a list of items. The main area shows a grid of dates with colored bars representing different planning entries.

Figure 3.18: Interface new planning

Update a planning:

Plannings could be updated by clicking on the update planning, Once that button is clicked a modal is shown with the old planning's details allowing the administrator to modify it, the modal is shown in figure <Figure 3.19>

The image shows a web application interface for updating a planning. A modal window titled "Mise a jour d'un planning" is centered on the screen. The modal contains several input fields and a list of items. The first field is labeled "Merchandiser" and contains the text "Mohamed". The second field is labeled "Choisir le merchandiser" and contains the text "Lundi". Below these fields is a list of items, each with a label, a delete button (X), and a dropdown arrow (v). The items are: "GMS-3", "GMS-10", "GMS-8", "GMS-4", "gms22", "GMS-2", "GMS-4", "GMS-7", "GMS-1", "GMS-7", "GMS-9", "gms22", "GMS-8", and "Dimanche". The modal is overlaid on a background that shows a calendar for June 2021 and a search bar.

Figure 3.19: Interface Update a planning

Delete a planning:

The administrator could delete any planning by clicking the delete planning button, the modal shown in figure<Figure 3.20> is displayed once that button is clicked, where the merchandiser whose planning to be deleted is selected.

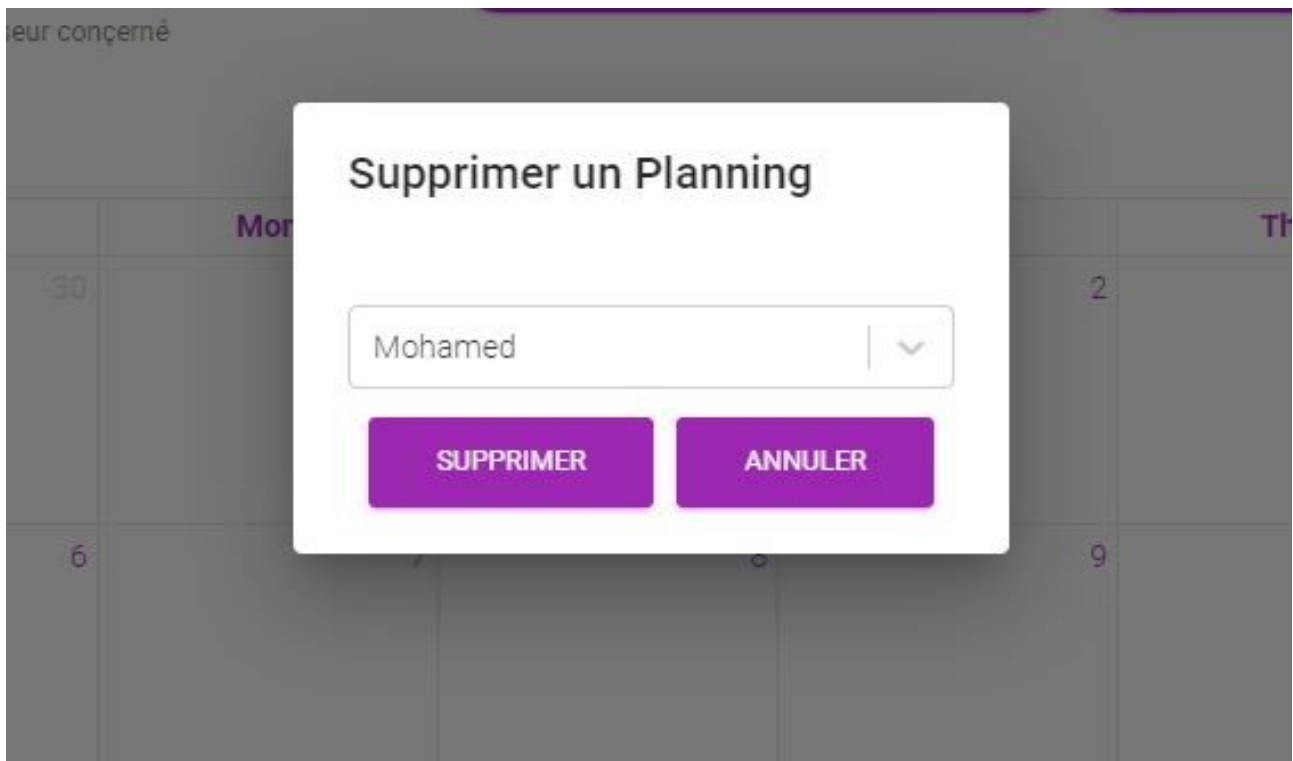
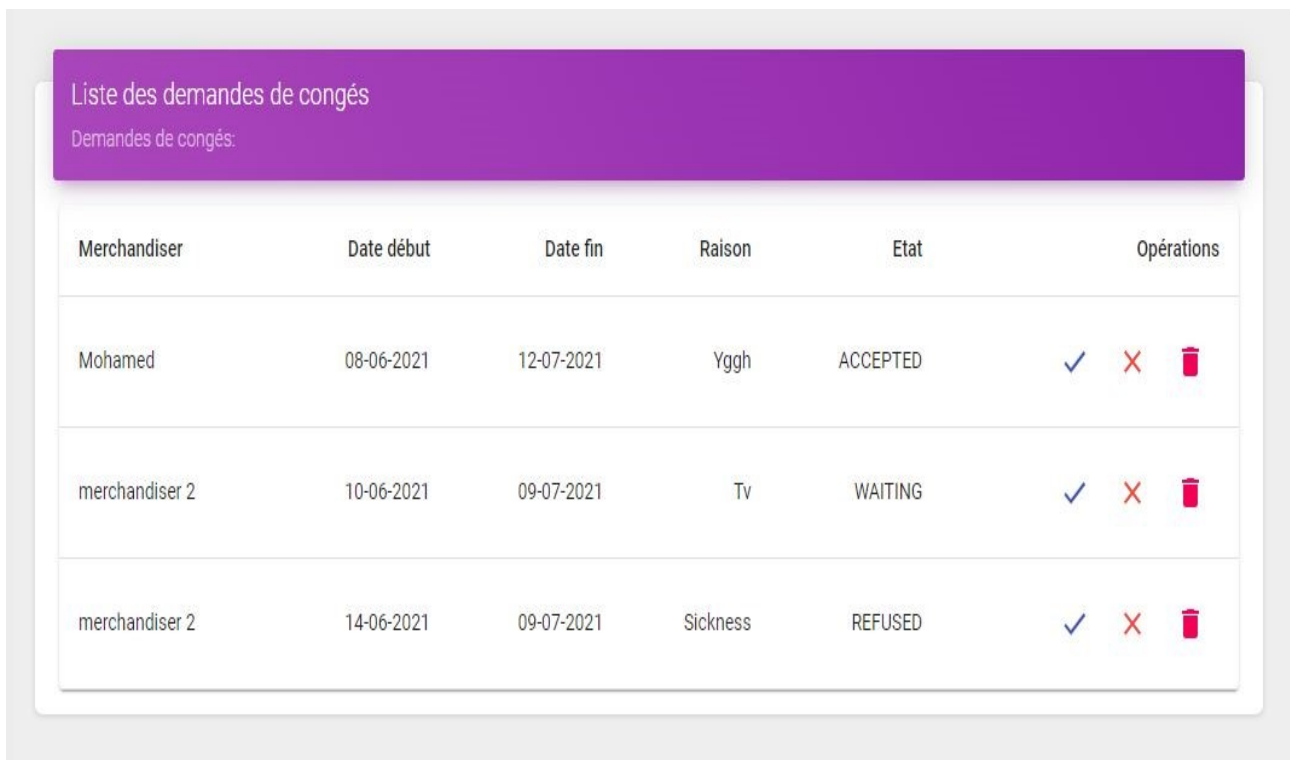


Figure 3.20: Interface Delete a planning

3.4.1.4 Leave requests management

Managing leave requests list:

By clicking on Leave request in the sidebar, the list of all leave requests is displayed in a table figure<>. By default the state of all leave requests is “WAITING”, the administrator has two icon buttons the change the state to “ACCEPTED ” or “REFUSED”.



The screenshot shows a web interface titled "Liste des demandes de congés" (List of leave requests). Below the title is a subtitle "Demandes de congés:". The main content is a table with the following columns: "Merchandiser", "Date début", "Date fin", "Raison", "Etat", and "Opérations". There are three rows of data. Each row in the "Opérations" column contains three icons: a blue checkmark, a red X, and a red trash can.

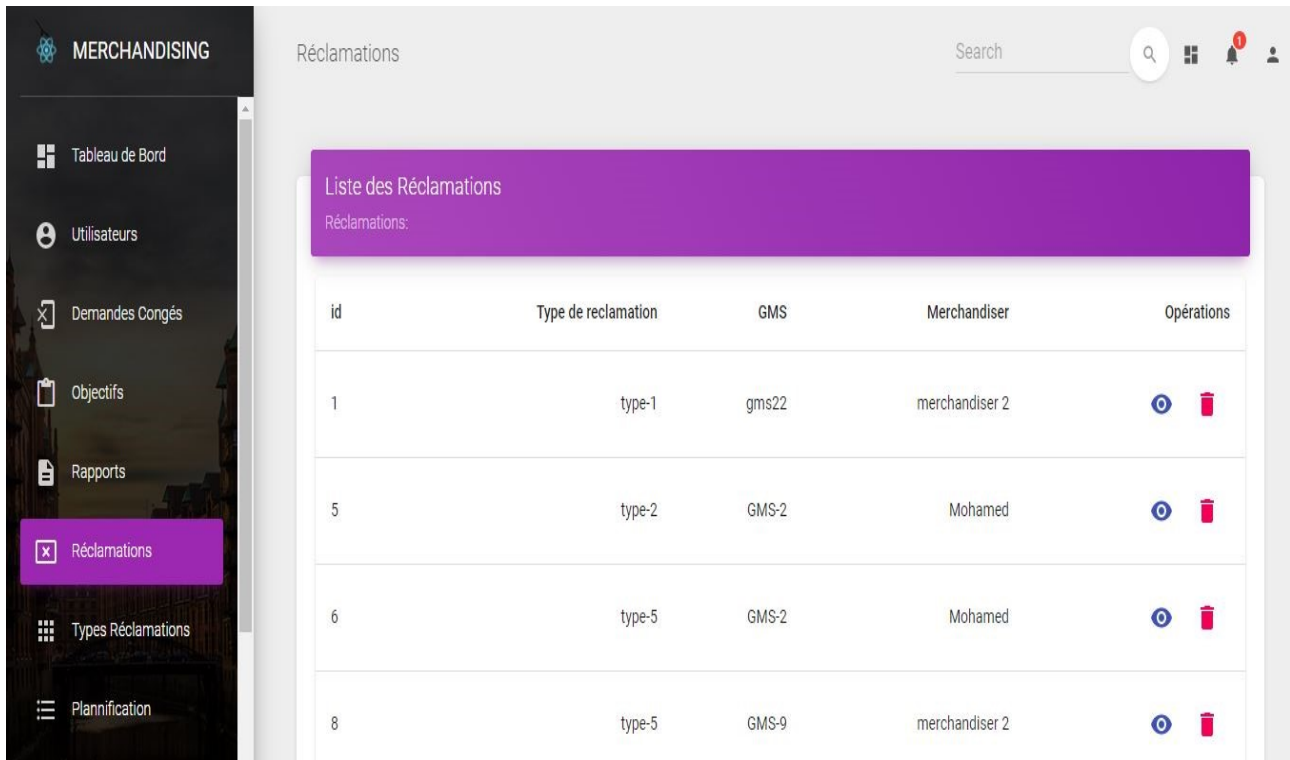
Merchandiser	Date début	Date fin	Raison	Etat	Opérations
Mohamed	08-06-2021	12-07-2021	Yggh	ACCEPTED	✓ ✗ 🗑️
merchandiser 2	10-06-2021	09-07-2021	Tv	WAITING	✓ ✗ 🗑️
merchandiser 2	14-06-2021	09-07-2021	Sickness	REFUSED	✓ ✗ 🗑️

Figure 3.21: Interface Managing leave requests list

3.4.1.5 Claims management

See the list of claims:

By clicking on Claims in the sidebar, the list of all claims is displayed in a table, figure<Figure 3.22>



id	Type de reclamation	GMS	Merchandiser	Opérations
1	type-1	gms22	merchandiser 2	 
5	type-2	GMS-2	Mohamed	 
6	type-5	GMS-2	Mohamed	 
8	type-5	GMS-9	merchandiser 2	 

Figure 3.22: Interface list of claims

See claim details:

By clicking at the eye icon in the operations section the Claim's details are displayed in a modal with an optional picture send from the merchandiser. The modal is shown in figure<Figure 3.23>

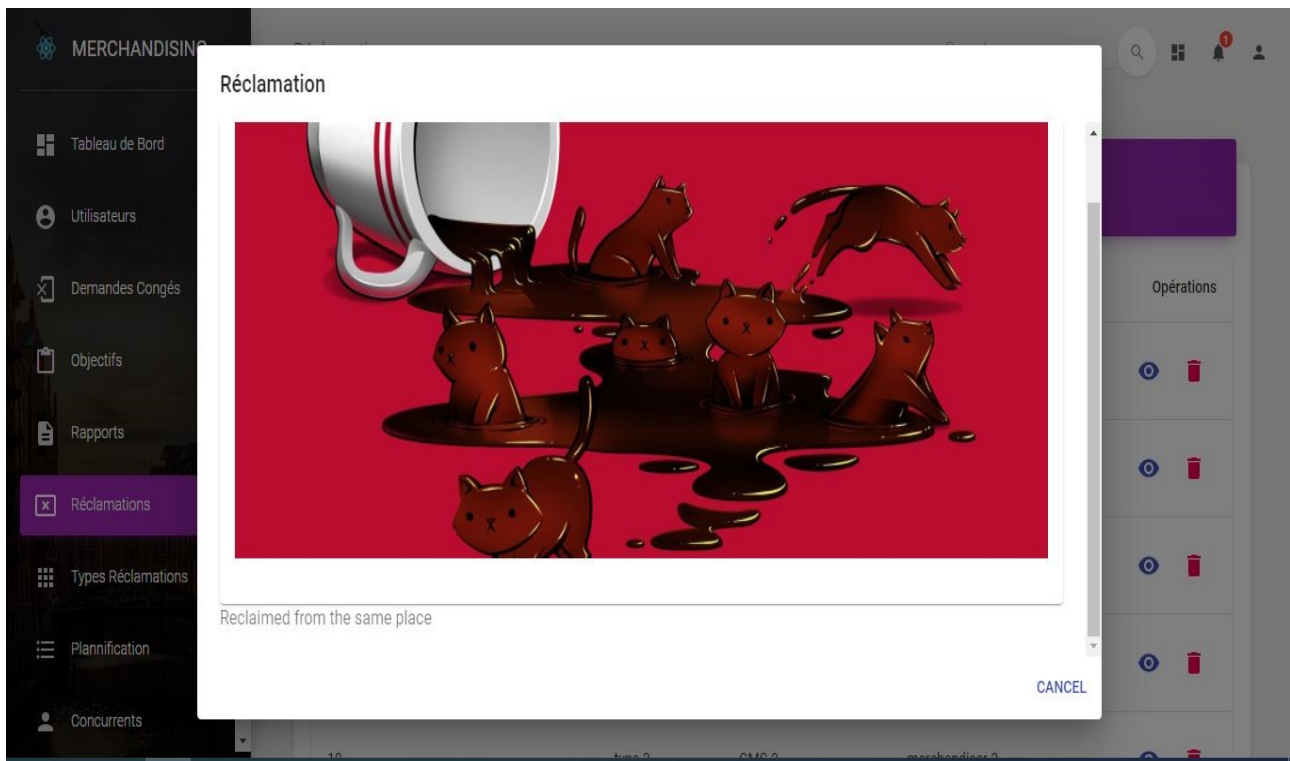


Figure 3.23: Interface claim details

Delete a Claim:

BThe administrator could delete any claim by clicking the delete icon in the operations section, the modal shown in figure<Figure 3.24> is displayed once that icon is clicked.

¹⁹Réalisation

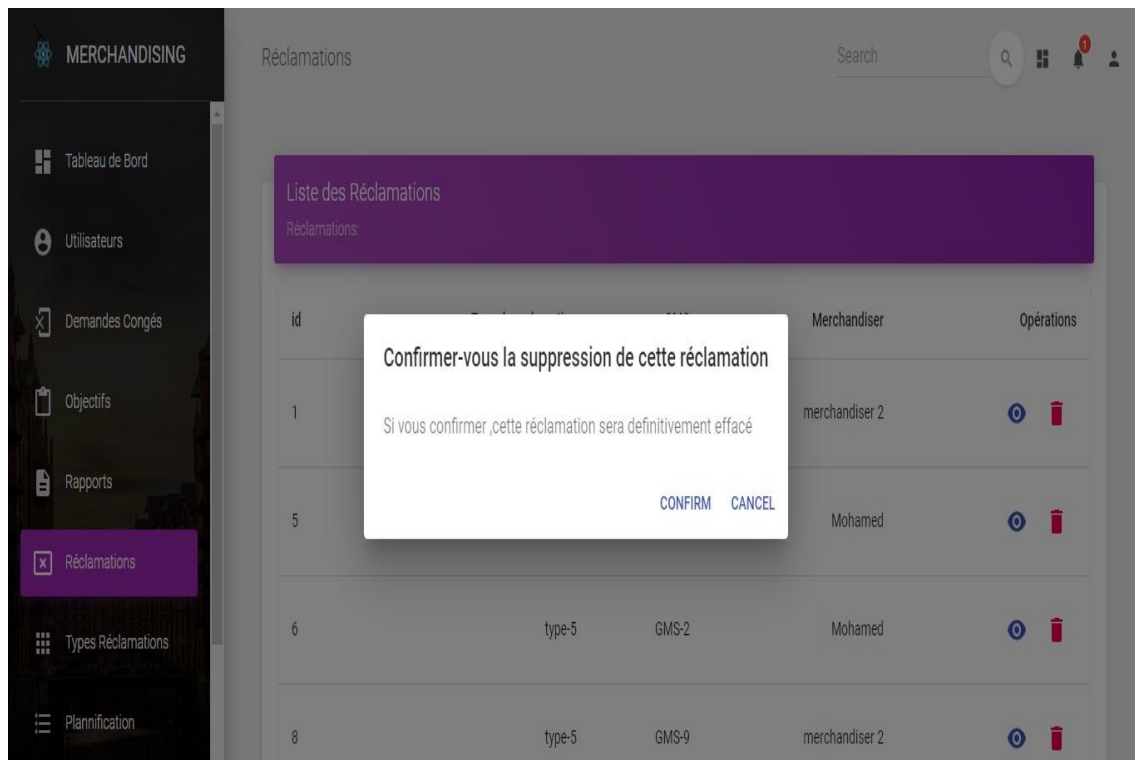


Figure 3.24: Interface Delete a Claim

3.4.1.6 Reports management

See the list of reports:

By clicking on Reports in the sidebar, the list of all reports is displayed in a table, figure<Figure 3.25>

id	Date	GMS	Merchandiser	Opérations
8	07-06-2021	GMS-6	Mohamed	
9	07-06-2021	GMS-7	Mohamed	
10	07-06-2021	GMS-8	Mohamed	
11	07-06-2021	GMS-6	Mohamed	

Figure 3.25: Interface list of reports

See report details:

By clicking on Reports in the sidebar, the list of all reports is displayed in a table, figure<Figure 3.26>

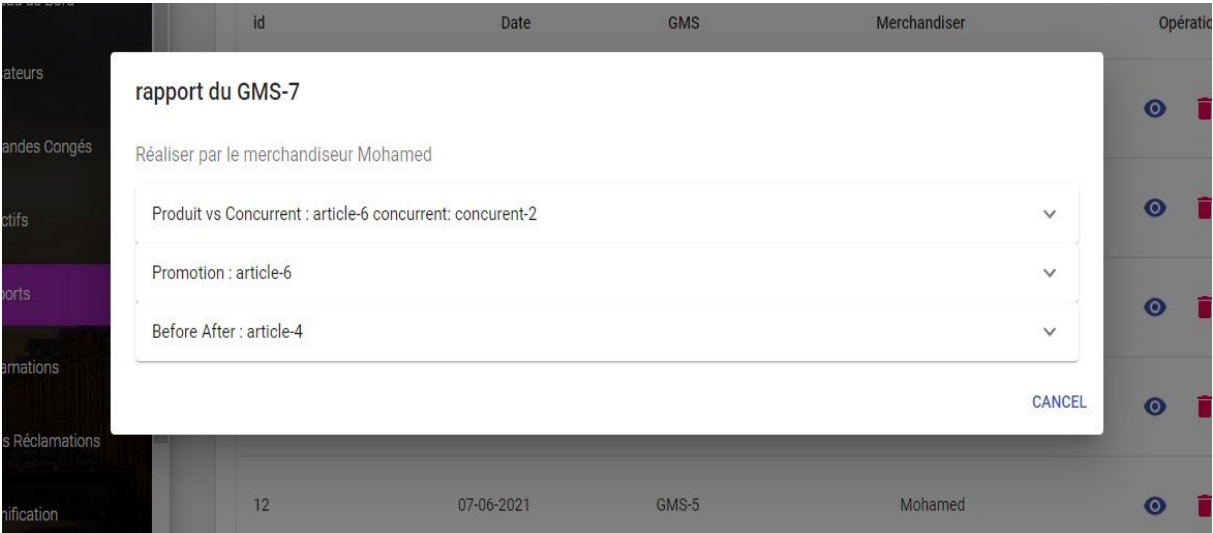


Figure 3.26: Interface report details

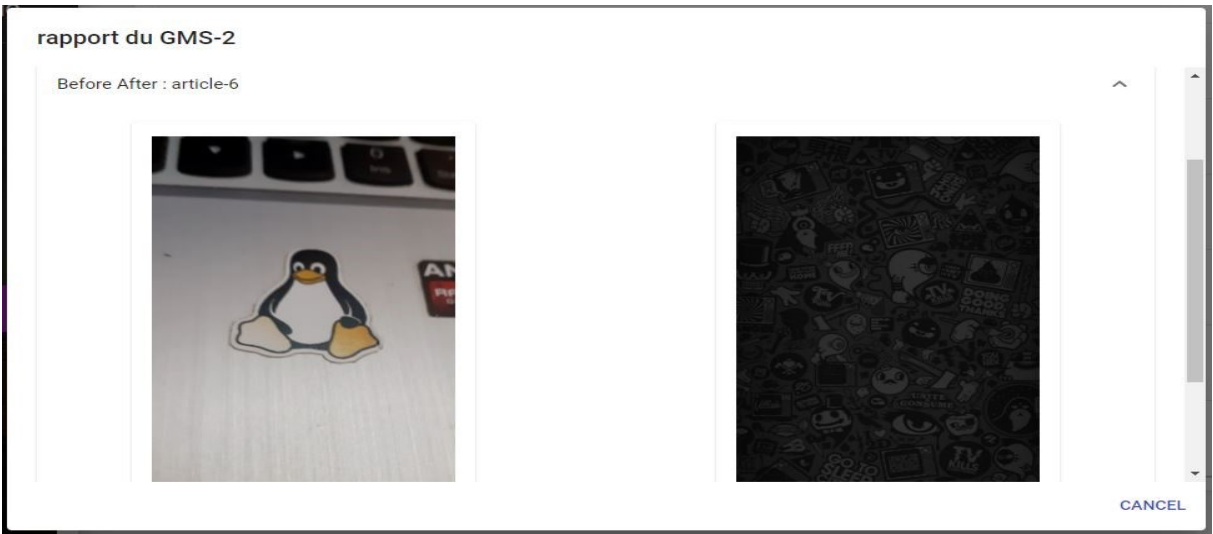


Figure 3.27: Interface event Before After in report details

²¹Réalisation

3.4.2 Merchandiser : (Developing the mobile app is the task of colleague)

3.4.2.1 Login

Providing the correct email and password allows the user to enter the app to the home screen. In case of entering the wrong credentials an error is shown.

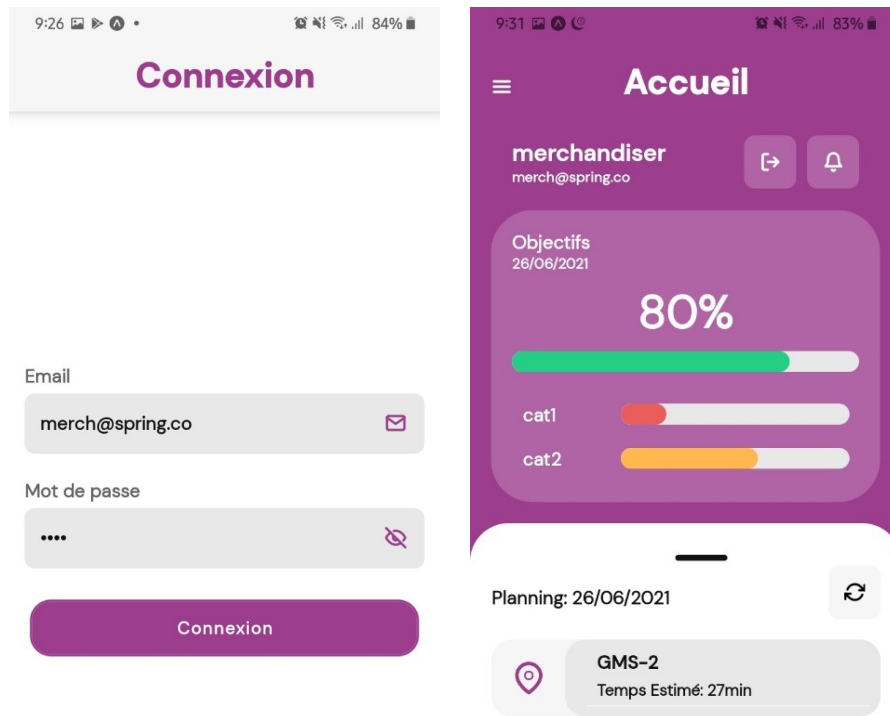


Figure 3.28: Login & HomePage

3.4.2.2 Locating a GMS

Before starting any task the merchandiser must first confirm his proximity to the selected supermarket. The map clearly shows the acceptable radius for the supermarket as well as the merchandiser's current location.

²²Réalisation

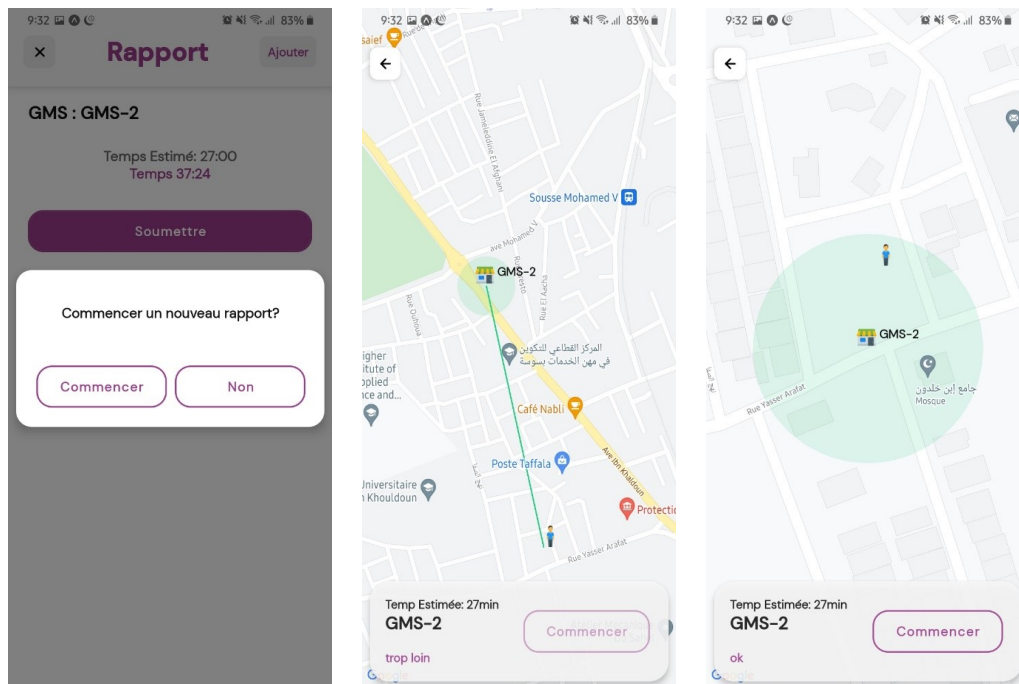


Figure 3.29: Locating a GMS to start a report

3.4.2.3 Creating a report

After selecting a task and starting a report, the user is redirected to the report screen where he can add events, fill the report form and when ready submit the report. Filling the report means filling the form with valid information for each event. Events differ in the shape and type of the required information and this is reflected in the displayed form.

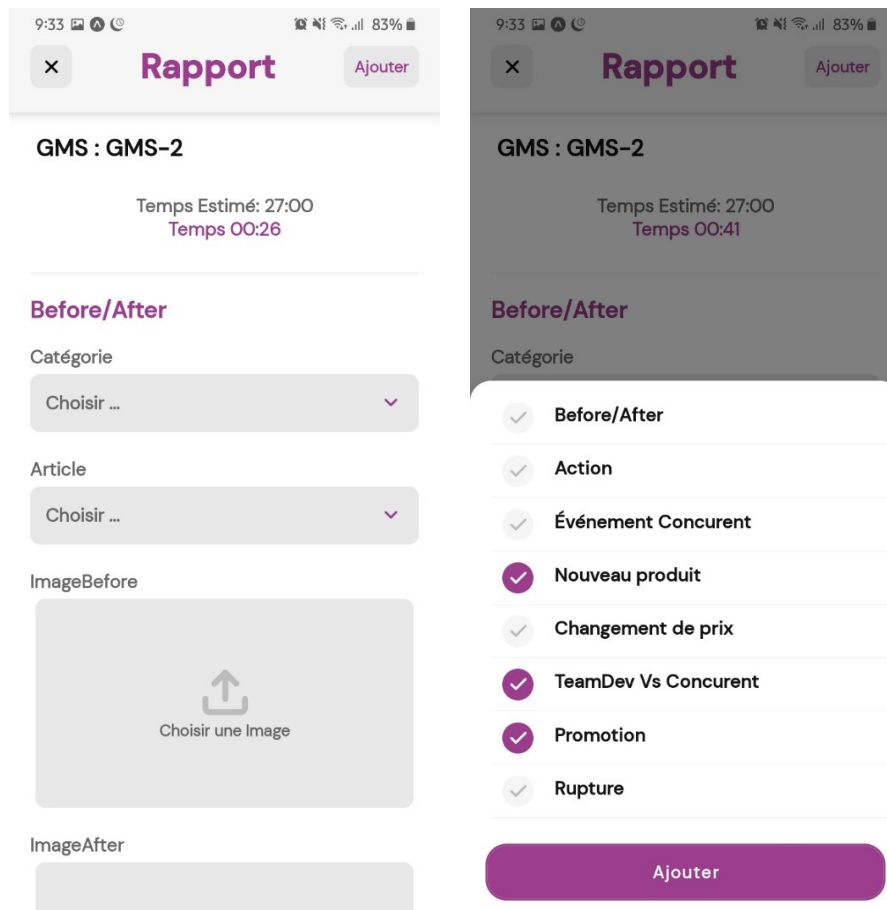


Figure 3.30: Creating a report and populating it with events

Conclusion

In this chapter, we presented our global physical and logical architecture, explained our choice of technologies and concluded by various screenshots of our solution.

CONCLUSION& PERSPECTIVE

During this end of studies project performed in the host company“TeamDev” we were able to create a solution named “merchandising team », a web and mobile application to digitalize merchandisers’ work, as a centralized solution to manage merchandisers, their reports, and their collected data.

The mobile application is used by the merchandiser to follow his planning, create reports, send claims and leave requests and reference the products in a given supermarket. As for the web application, it is used by the administrator as a centralized solution for managing and monitoring all of the merchandiser team’s activities

This report details in its 3 chapters the various parts in the process of creating our solution starting with the state of the art where we did our preliminary study, suggested our solution and specified various functional and non-functional requirements.

The second chapter deals with the conception, explaining the behavior, interactions and structure of our solution while the third chapter’s main focus was the realization. In this last part we explained the used architecture and the chosen technologies along with screenshots of the major parts of the application.

While coding I was able to learn new technologies, tools and design patterns.

¹Conclusion

Several third party libraries were used too such as “formik” for form handling in react and “full calendar” as a JavaScript package to manipulate calendars.

Certainly, this work achieved most of its objectives. But there is still work to be done. One of the most important features that could be beneficial for our application is creating a tool to share documents between the administrator and merchandisers that could be used mainly to share announcements, and training materials or guidelines.

Another feature to be added is the possibility to create in store audits where the administrator could create those audits dynamically and send them to specific merchandisers to be executed. Data from those audits are collected in real time.

A tool to optimize the routes between the supermarkets that compose the planning could be a good idea to minimize fuel consumption and save time.

²Conclusion

WEBOGRAPHY

- [1] <https://bi-survey.com/decision-making-no-information>.visited on 23/02/2021.
- [2] <https://www.geeksforgeeks.org/unified-modeling-language-uml-class-diagrams/> .visited on 20/02/2021.
- [3] <https://www.geeksforgeeks.org/unified-modeling-language-uml-sequence-diagrams/> .visited on 20/03/2021.
- [4] <https://spring.io/projects/spring-boot> .visited on 15/06/2021.
- [5] <https://reactjs.org/>.visited on 15/06/2021.
- [6] <https://www.postgresql.org/media/img/about/press/elephant.png> .visited on 15/06/2021.
- [7] <https://www.postgresql.org/about/> .visited on 15/06/2021.
- [8] <https://code.visualstudio.com/docs/editor/whyvscode> .visited on 15/06/2021.
- [9] <https://www.baeldung.com/eclipse-sts-spring> .visited on 15/06/2021.
- [10] <https://about.gitlab.com/press/press-kit/>.visited on 15/06/2021.
- [11] <https://www.computerhope.com/jargon/d/drawio.htm>.visited on 08/03/2021.
- [12] <https://stackshare.io/beekeeper-studio/:text=what%20is%20beekeeper%20studio%3f,%2c%20sqlite%20> on 12/03/2021.

- [13] <https://www.postgresql.org/> .visited on 15/06/2021.
- [14] <https://spring.io/tools> .visited on 15/06/2021.
- [15] <https://www.geeksforgeeks.org/introduction-postman-api-development/> .visited on 15/06/2021.
- [16] <https://www.postman.com/company/press-media/> .visited on 15/06/2021.
- [17] <https://whatistechtarget.com/definition/gitlab>.visited on 15/06/2021.

ANNEX A